Fine-Grained GPU Sharing for Containerised Inference

Technical implementation guide for the Inference Workload Management Runtime (IWMR) on NVIDIA (MPS) and AMD (ROCm CU masking): host setup, container contract, runtime modules, commands and test plan

Ioannis Kyriakopoulos · Industrial Systems Institute, Athena Research Center · 10 November 2026 · revision 2
Companion document: *Architecture Overview*. Reference implementation: the `gpushare/` source tree delivered with this guide.

§	Section	§	Section
1	Scope, assumptions and versions	6	Inference container contract
2	Workload request	7	The IWMR runtime container
3	Source tree	8	Test and acceptance plan
4	NVIDIA host setup (MPS)	9	Operations command reference
5	AMD host setup (CU masking)	10	Limitations and items to verify
		A	Appendix: test scripts

1. Scope, assumptions and versions

The guide specifies how to build the system described in the Architecture Overview. Inference models run in Docker containers on shared GPUs; each container receives a compute and a memory ceiling fixed at start. A single container per GPU node, the Inference Workload Management Runtime (IWMR), performs all management: API, registry, admission with deterministic best-fit placement, launching, request routing, exit and fault monitoring, recovery, telemetry and image validation. Every component is given as a working script with the commands that install it. The code is a reference implementation for the COGAC testbed and favours clarity over hardening (no authentication, in-memory state).

Assumptions:

- One vendor per node and one IWMR per node. Nodes are independent; multi-node coordination is out of scope.
- Linux hosts with Docker Engine and systemd.
- One container serves one model on one GPU; requests reach containers only through the IWMR router.
- Model weights are immutable and mounted read-only, so recovery never needs to save GPU state.
- Nothing outside the IWMR launches GPU work on the shared GPUs (otherwise the ledger's accounting is wrong).

Item	NVIDIA	AMD
GPU generation	Volta or newer (per-client address spaces under MPS)	CDNA (MI100/MI200/MI300); partitioning needs CDNA 3 (MI300) or newer
Driver / runtime	Driver supporting CUDA ≥ 11.5 (pinned memory limit). COGAC driver reports CUDA 12.6: sufficient	ROCm 6.x or 7.x with amdgpu kernel driver
Container runtime	nvidia-container-toolkit (--gpus)	Plain Docker with /dev/kfd and /dev/dri/renderD* passed through
Compute ceiling	CUDA_MPS_ACTIVE_THREAD_PERCENTAGE	HSA_CU_MASK (explicit CU range)

Item	NVIDIA	AMD
Memory ceiling	CUDA_MPS_PINNED_DEVICE_MEM_LIMIT	libhipmemcap.so via LD_PRELOAD (this guide)
Image validation	compute-sanitizer from the host CUDA toolkit	AddressSanitizer build of the image (xnack+)
Telemetry	nvidia-smi	sysfs: gpu_busy_percent, mem_info_vram_used
Hard isolation	MIG; MPS inside a MIG slice	CPX/DPX compute + NPS2/NPS4 memory partitions
IWMR image	python:3.12-slim + fastapi, uvicorn, docker, requests (no CUDA, no PyTorch)	same image

Table 1. Platform requirements.

2. Workload request

A workload is registered with one JSON document posted to the IWMR. The vendor backend translates the vendor-neutral fields into container settings. On NVIDIA a share is a *ceiling* on the fraction of SMs a client may occupy, with the hardware choosing which SMs; on AMD the backend assigns a *concrete, disjoint* set of CUs.

deploy/example_request.json

```
{
  "workload_id": "llm-a-001",
  "image": "registry.local/infer/llama-8b:1.4",
  "image_digest": "registry.local/infer/llama-8b@sha256:<digest>",
  "compute_share": 0.25,
  "mem_bytes": 12884901888,
  "isolation": "shared",
  "port": 8000,
  "infer_path": "/infer",
  "env": {
    "MODEL_PATH": "/models/model.pt",
    "MAX_BATCH": "32"
  },
  "volumes": {
    "/srv/models/llama-8b": "/models"
  }
}
```

Field	Meaning	NVIDIA translation	AMD translation
workload_id	Unique name; container is gs-<id>	—	—
image, image_digest	Image and its content address	digest must be approved for shared placement	same
compute_share	Fraction of the GPU, 0–1	ceil(share·100) → CUDA_MPS_ACTIVE_THREAD_PERCENTAGE (MPS rounds to TPCs)	n = ceil(share·CUs), even → HSA_CU_MASK=0:first-last
mem_bytes	Device-memory ceiling	CUDA_MPS_PINNED_DEVICE_MEM_LIMIT =0=<MiB>M	HIPMEM_LIMIT_BYTES + LD_PRELOAD=libhipmemcap.so
isolation	shared (default) | exclusive	exclusive: alone on a GPU; allowed without validation	same
port, infer_path	Where the server listens	router target http://gs-<id>:port/infer_path	same
env, volumes, command	Passed to the container; volumes mounted read-only	—	—

Table 2. Request fields (validated in runtime.py, translated in runtime/backends/).

3. Source tree

All Python modules compile; the runtime was exercised offline with Docker, the GPU backend and the inference containers replaced by stubs (submission, best-fit placement, generated MPS settings, request routing, a culprit/victim incident and an OOM re-placement), and the memory-cap library was built and tested against a stub HIP library. Nothing was executed on a GPU; Section 8 is the acceptance procedure for real hardware.

File	Runs in	Purpose
deploy/setup_nvidia_node.sh, nvidia-mps@.service	NVIDIA host	Persistence + EXCLUSIVE_PROCESS, one MPS daemon per GPU, validation GPU left without MPS
deploy/setup_amd_node.sh	AMD host	Build libhipmemcap.so; print CU/XCC/VRAM inventory
deploy/run_runtime.sh	Host	Create the Docker network and start the IWMR container with its privileges
deploy/example_request.json	Client	Example workload registration
runtime/Dockerfile	Build host	IWMR image
runtime/runtime.py	IWMR	REST API, registry, launcher, exit watcher, health watcher, start-up
runtime/ledger.py	IWMR	Admission ledger and best-fit placement
runtime/backends/{base,nvidia,amd}.py	IWMR	Vendor translation of shares and memory caps
runtime/router.py	IWMR	Inference request router with in-flight journal and replay
runtime/faults.py	IWMR	/dev/kmsg watcher: Xid / amdgpu faults → container
runtime/recovery.py	IWMR	Incident grouping; culprit dropped, victims re-placed
runtime/telemetry.py	IWMR	Utilisation and memory per GPU
runtime/validator.py	IWMR	Memory-checker runs on the validation GPU; approved digests
container/guard.py, oom_handling.py, Dockerfile.example, smoke_test_template.py	Inference image	Error catcher, recoverable OOM, image contract, validation inputs
container/hipmemcap.c	AMD inference containers	LD_PRELOAD memory cap for hipMalloc/hipFree (+ async)
tests/*.py	Host or container	SM count, burn load (NVIDIA/AMD), memory cap, fault injection

Table 3. Source tree.

```
gpushare/
├── deploy/     setup_nvidia_node.sh  nvidia-mps@.service  setup_amd_node.sh  run_runtime.sh  example_request.json
├── runtime/    Dockerfile  runtime.py  ledger.py  router.py  faults.py  recovery.py  telemetry.py  validator.py
│   └── backends/  base.py  nvidia.py  amd.py
├── container/  guard.py  oom_handling.py  hipmemcap.c  Dockerfile.example  smoke_test_template.py
└── tests/      gpu_lowlevel.py  mps_sm_count.py  mps_burn.py  hip_burn.py  mem_cap_test.py  fault_inject.py
```

Reference architecture

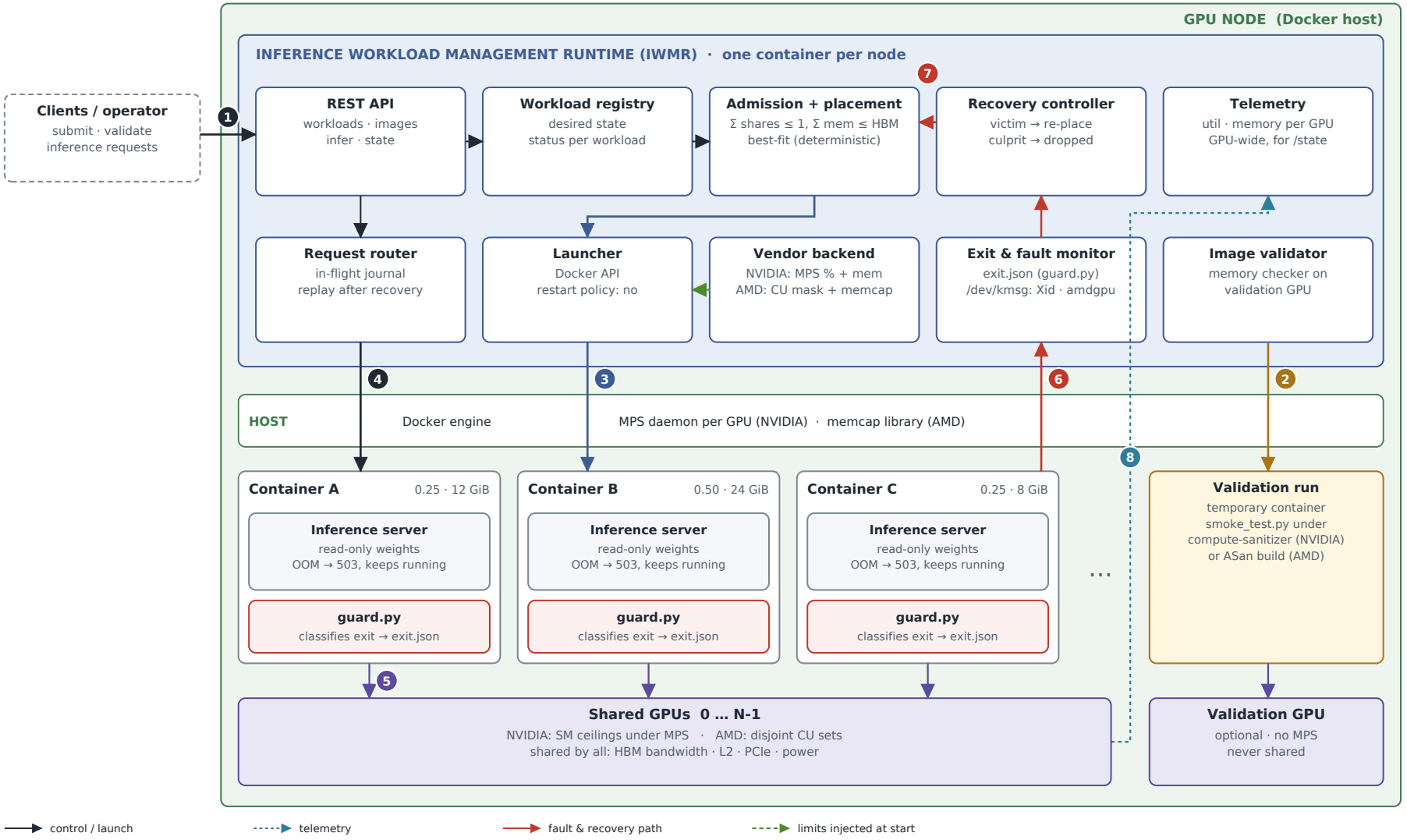


Figure 1. One GPU node, as in the Architecture Overview. Host setup: §4–5; inference containers: §6; IWMR modules: §7.

4. NVIDIA host setup (MPS)

4.1 Prerequisites

```
nvidia-smi                                  # driver loaded; note the CUDA version in the header
nvidia-cuda-mps-control -h >/dev/null && echo "MPS available"
docker run --rm --gpus all ubuntu nvidia-smi -L   # NVIDIA container runtime works
# if not: install nvidia-container-toolkit, then
sudo nvidia-ctk runtime configure --runtime=docker && sudo systemctl restart docker
```

4.2 One MPS daemon per GPU

The MPS daemons are the only management-related processes outside the IWMR, because they are part of the driver stack. A single daemon serving several GPUs would make them one fault domain, so the template unit starts one daemon per GPU, each with its own pipe and log directory under `/var/run/nvidia-mps/gpuN`. Clients select the daemon (and GPU) through `CUDA_MPS_PIPE_DIRECTORY`. EXCLUSIVE_PROCESS mode ensures only the MPS server owns the GPU, so no process can bypass the limits. The validation GPU, if any, is left in DEFAULT mode without MPS because compute-sanitizer cannot run as an MPS client.

`deploy/nvidia-mps@.service`

```
# /etc/systemd/system/nvidia-mps@.service
# One MPS control daemon per GPU; instance name = GPU index (e.g. nvidia-mps@0).
[Unit]
Description=NVIDIA MPS control daemon for GPU %i
After=nvidia-persistenced.service

[Service]
Type=forking
Environment=CUDA_VISIBLE_DEVICES=%i
Environment=CUDA_MPS_PIPE_DIRECTORY=/var/run/nvidia-mps/gpu%i
Environment=CUDA_MPS_LOG_DIRECTORY=/var/log/nvidia-mps/gpu%i
ExecStartPre=/bin/mkdir -p /var/run/nvidia-mps/gpu%i /var/log/nvidia-mps/gpu%i
ExecStart=/usr/bin/nvidia-cuda-mps-control -d
ExecStop=/bin/sh -c 'echo quit | /usr/bin/nvidia-cuda-mps-control'
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

`deploy/setup_nvidia_node.sh`

```
#!/usr/bin/env bash
# setup_nvidia_node.sh - prepare an NVIDIA GPU node for MPS-based sharing.
# Starts one MPS control daemon PER GPU, so each GPU is its own fault domain.
# Usage: sudo ./setup_nvidia_node.sh                    (all GPUs)
#        sudo ./setup_nvidia_node.sh 0 2                (selected GPUs)
#        sudo VALIDATION_GPU=3 ./setup_nvidia_node.sh   (GPU 3 kept without MPS)
set -euo pipefail

MPS_ROOT=/var/run/nvidia-mps
LOG_ROOT=/var/log/nvidia-mps

command -v nvidia-smi >/dev/null || { echo "nvidia-smi not found"; exit 1; }
command -v nvidia-cuda-mps-control >/dev/null || { echo "MPS control binary not found"; exit 1; }
docker info 2>/dev/null | grep -qi nvidia || echo "WARN: NVIDIA container runtime not detected (install nvidia-
  container-toolkit)"

GPUS=("$@")
if [ ${#GPUS[@]} -eq 0 ]; then
  mapfile -t GPUS < <(nvidia-smi --query-gpu=index --format=csv,noheader)
fi

for g in "${GPUS[@]}"; do
  if [ "$g" = "${VALIDATION_GPU:-}" ]; then
    nvidia-smi -i "$g" -c DEFAULT              # validation GPU: no MPS, checker needs it
    echo "GPU $g reserved for image validation"; continue
  fi
  echo "== GPU $g =="
  nvidia-smi -i "$g" -pm 1                    # persistence mode
  nvidia-smi -i "$g" -c EXCLUSIVE_PROCESS     # only the MPS server may own the GPU
  mkdir -p "$MPS_ROOT/gpu$g" "$LOG_ROOT/gpu$g"
  systemctl enable --now "nvidia-mps@$g.service"
done
```

```
for g in "${GPUS[@]}"; do
  [ "$g" = "${VALIDATION_GPU:-}" ] && continue
  CUDA_MPS_PIPE_DIRECTORY="$MPS_ROOT/gpu$g" \
    bash -c 'echo get_default_active_thread_percentage | nvidia-cuda-mps-control' \
    && echo "GPU $g: MPS daemon reachable"
done

sudo cp deploy/nvidia-mps@.service /etc/systemd/system/ && sudo systemctl daemon-reload
sudo VALIDATION_GPU=3 ./deploy/setup_nvidia_node.sh     # or without VALIDATION_GPU: all GPUs shared
systemctl status nvidia-mps@0
```

4.3 Manual client test (before starting the IWMR)

```
docker run --rm --gpus device=0 --ipc=host \
  -v /var/run/nvidia-mps/gpu0:/var/run/nvidia-mps/gpu0 \
  -e CUDA_MPS_PIPE_DIRECTORY=/var/run/nvidia-mps/gpu0 \
  -e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=25 \
  -e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0=4096M \
  -v $PWD/tests:/tests:ro  python:3.12-slim  python3 /tests/mps_sm_count.py
# expected: about 25 % of the GPU's SMs (rounded to whole TPCs)
```

4.4 Rules that are easy to get wrong

- `--ipc=host` and the mounted pipe directory are both required; without them a client runs without MPS (and without limits) or fails because the GPU is in EXCLUSIVE_PROCESS mode.
- Clients must run as the same UID as the MPS daemon (root here). An MPS server serves one UID.
- The device index in `CUDA_MPS_PINNED_DEVICE_MEM_LIMIT` is the index inside the container: with one GPU passed through it is `0`.
- Limits are read when the client creates its CUDA context; resizing means a restart.
- The SM percentage is rounded to whole TPCs; the backend records the share it requested, and `tests/mps_sm_count.py` shows the SMs actually granted.
- Each client's context uses device memory outside its limit; measure it and set `CTX_OVERHEAD_MB` for the IWMR (default 512).
- At most 48 clients per MPS server on Volta-class GPUs (MAX_CLIENTS_PER_GPU); check your CUDA version.

4.5 Optional: MIG plus MPS

On MIG-capable GPUs each MIG instance can host its own MPS daemon, so a fault only affects clients in that slice. MIG layouts require root and an idle GPU to change, so they are static host configuration.

```
sudo nvidia-smi -i 0 -mig 1                # enable MIG (GPU idle; may need reset)
sudo nvidia-smi mig -i 0 -lgip             # list instance profiles
sudo nvidia-smi mig -i 0 -cgi 9,9 -C       # e.g. two 3g slices with compute instances
nvidia-smi -L                              # slices appear as MIG-<uuid> devices
```

5. AMD host setup (CU masking)

AMD GPUs run kernels from several processes concurrently through hardware queues, so no daemon is needed. Compute is divided with CU masks; device memory has no driver-level per-process cap, which the library of Section 5.3 provides. The host only needs ROCm and the compiled library.

5.1 Prerequisites and inventory

```
ls -l /dev/kfd /dev/dri/renderD*         # ROCm device nodes
amd-smi version && amd-smi list
rocminfo | grep -E "Marketing Name|Compute Unit"
sudo ./deploy/setup_amd_node.sh            # builds /opt/gpushare/lib/libhipmemcap.so, prints inventory
```

deploy/setup_amd_node.sh

```
#!/usr/bin/env bash
# setup_amd_node.sh - prepare an AMD (ROCm) GPU node for CU-mask based sharing.
# Builds the hipMalloc interposer and prints the GPU inventory from KFD sysfs.
# Usage: sudo ./setup_amd_node.sh
set -euo pipefail

LIB_DIR=/opt/gpushare/lib
SRC_DIR="$(cd "$(dirname "$0")/../container" && pwd)"

[ -e /dev/kfd ] || { echo "/dev/kfd missing: amdgpu/ROCm driver not loaded"; exit 1; }
command -v amd-smi >/dev/null || echo "WARN: amd-smi not found (monitoring/partitioning unavailable)"

mkdir -p "$LIB_DIR"
gcc -O2 -fPIC -shared -o "$LIB_DIR/libhipmemcap.so" "$SRC_DIR/hipmemcap.c" -ldl -lpthread
echo "built $LIB_DIR/libhipmemcap.so"

echo "== GPU inventory (KFD topology) =="
for n in /sys/class/kfd/kfd/topology/nodes/*; do
  p="$n/properties"
  simd=$(awk '/^simd_count/{print $2}' "$p")
  [ "${simd:-0}" -gt 0 ] || continue            # skip CPU nodes
  per_cu=$(awk '/^simd_per_cu/{print $2}' "$p")
  minor=$(awk '/^drm_render_minor/{print $2}' "$p")
  xcc=$(awk '/^num_xcc/{print $2}' "$p"); xcc=${xcc:-1}
  vram=$(cat "/sys/class/drm/renderD$minor/device/mem_info_vram_total" 2>/dev/null || echo 0)
  echo "renderD$minor  CUs=$((simd / per_cu))  XCCs=$xcc  VRAM=$((vram / 1024 / 1024)) MiB"
done

if command -v amd-smi >/dev/null; then
  amd-smi static --partition 2>/dev/null || true
fi
```

5.2 Compute: HSA_CU_MASK

`HSA_CU_MASK` is applied by the ROCm runtime when it creates hardware queues, so it covers every kernel of the process. The format is `<device>:<CU list>`, e.g. `0:0-75`; with one GPU visible in the container the device index is 0. The backend allocates contiguous, even-sized, disjoint ranges, preferring ranges inside one XCC (one die on MI300), and tries the next GPU when a range does not fit.

Verify on hardware: the mapping from CU index in the mask to physical XCC is architecture-dependent. Before relying on XCC-local ranges, record the hardware IDs reached by a masked kernel and confirm that a range chosen by the backend lands on one XCC. Adjust _find_range in runtime/backends/amd.py if not.

5.3 Memory: hipMalloc interposer

The library is preloaded into the inference process. It forwards allocations to the real HIP runtime, keeps a table of live blocks and returns hipErrorOutOfMemory once the live total would exceed `HIPMEM_LIMIT_BYTES`. Frameworks see an ordinary out-of-memory error. Tested against a stub HIP library: with a 1 GiB cap a second 600 MB allocation is denied and succeeds again after the first block is freed.

container/hipmemcap.c

```
/* hipmemcap.c - per-process device-memory cap for HIP applications (AMD).
 *
 * Loaded with LD_PRELOAD. Wraps hipMalloc/hipFree (and the async variants),
 * tracks live allocations and returns hipErrorOutOfMemory once the total would
 * exceed HIPMEM_LIMIT_BYTES. Plays the role of CUDA_MPS_PINNED_DEVICE_MEM_LIMIT.
 *
 * Build:  gcc -O2 -fPIC -shared -o libhipmemcap.so hipmemcap.c -ldl -lpthread
 * Use:    LD_PRELOAD=/opt/gpushare/lib/libhipmemcap.so HIPMEM_LIMIT_BYTES=8589934592 app
 *
 * Limits: allocations made through other entry points (hipMallocManaged,
```

```
 * hipExtMallocWithFlags, hipMallocPitch, symbols resolved via hipGetProcAddress)
 * are not counted. Verify coverage for each framework with tests/mem_cap_test.py.
 */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef int hipError_t;
typedef void *hipStream_t;
#define hipSuccess 0
#define hipErrorOutOfMemory 2

#define TABLE_SIZE (1 << 16)                /* open-addressing table of live blocks */
static void *keys[TABLE_SIZE];
static size_t sizes[TABLE_SIZE];
static size_t used = 0, limit = 0;
static pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER;

static void init_limit(void) {
    if (limit) return;
    const char *s = getenv("HIPMEM_LIMIT_BYTES");
    limit = s ? strtoull(s, NULL, 10) : SIZE_MAX;
    if (!limit) limit = SIZE_MAX;
}

static size_t slot(void *p) { return ((uintptr_t)p >> 8) & (TABLE_SIZE - 1); }

static void track(void *p, size_t n) {
    for (size_t i = slot(p), k = 0; k < TABLE_SIZE; i = (i + 1) & (TABLE_SIZE - 1), k++)
        if (!keys[i] || keys[i] == (void *)1) { keys[i] = p; sizes[i] = n; return; }
}

static size_t untrack(void *p) {
    for (size_t i = slot(p), k = 0; k < TABLE_SIZE && keys[i]; i = (i + 1) & (TABLE_SIZE - 1), k++)
        if (keys[i] == p) { keys[i] = (void *)1; return sizes[i]; }   /* tombstone */
    return 0;
}

static int reserve(size_t n) {              /* 1 = allowed, 0 = over the cap */
    int ok;
    pthread_mutex_lock(&mu);
    init_limit();
    ok = (used + n <= limit);
    if (ok) used += n;
    pthread_mutex_unlock(&mu);
    if (!ok) fprintf(stderr, "[hipmemcap] denied %zu B (used %zu / limit %zu)\n", n, used, limit);
    return ok;
}

static void commit(void *p, size_t n, hipError_t rc) {
    pthread_mutex_lock(&mu);
    if (rc == hipSuccess && p) track(p, n); else used -= n;
    pthread_mutex_unlock(&mu);
}

static void release(void *p) {
    pthread_mutex_lock(&mu);
    used -= untrack(p);
    pthread_mutex_unlock(&mu);
}

typedef hipError_t (*malloc_fn)(void **, size_t);
typedef hipError_t (*free_fn)(void *);
typedef hipError_t (*malloc_async_fn)(void **, size_t, hipStream_t);
typedef hipError_t (*free_async_fn)(void *, hipStream_t);

/* Resolve the real HIP symbol (next in link order) once per wrapper. */
#define REAL(name, type) static type real_##name; \
    if (!real_##name) real_##name = (type)dlsym(RTLD_NEXT, #name)

hipError_t hipMalloc(void **ptr, size_t n) {
    REAL(hipMalloc, malloc_fn);
    if (!reserve(n)) return hipErrorOutOfMemory;
    hipError_t rc = real_hipMalloc(ptr, n);
    commit(rc == hipSuccess ? *ptr : NULL, n, rc);
    return rc;
}

hipError_t hipFree(void *p) {
    REAL(hipFree, free_fn);
```

```
    hipError_t rc = real_hipFree(p);
    if (rc == hipSuccess && p) release(p);
    return rc;
}

hipError_t hipMallocAsync(void **ptr, size_t n, hipStream_t s) {
    REAL(hipMallocAsync, malloc_async_fn);
    if (!reserve(n)) return hipErrorOutOfMemory;
    hipError_t rc = real_hipMallocAsync(ptr, n, s);
    commit(rc == hipSuccess ? *ptr : NULL, n, rc);
    return rc;
}

hipError_t hipFreeAsync(void *p, hipStream_t s) {
    REAL(hipFreeAsync, free_async_fn);
    hipError_t rc = real_hipFreeAsync(p, s);
    if (rc == hipSuccess && p) release(p);
    return rc;
}

LD_PRELOAD=/opt/gpushare/lib/libhipmemcap.so HIPMEM_LIMIT_BYTES=$((2*1024**3)) \
  python3 tests/mem_cap_test.py amd          # expect ~2.00 GiB before the first failure
```

5.4 Optional: hard partitions (MI300 and newer)

```
sudo amd-smi partition --accelerator          # profiles and supported NPS modes
sudo amd-smi set --gpu all --compute-partition CPX
sudo amd-smi set --memory-partition NPS4        # MI300X: NPS4 requires CPX
amd-smi static --partition                      # verify; partitions appear as separate GPUs
```

6. Inference container contract

Any inference image can be used if (i) its entrypoint is guard.py wrapping the server; (ii) it contains /app/smoke_test.py for validation; (iii) model weights come from a read-only volume; (iv) the server exposes /health (used by the IWMR for readiness) and handles OOM as a recoverable error. The image never sets GPU limits; the IWMR injects them.

6.1 guard.py: the error catcher

guard.py runs the server as a child, mirrors its stderr, keeps the last 200 lines and, on exit, classifies the cause and atomically writes `/run/gpushare/exit.json` into a per-workload host directory that the IWMR reads. Victim patterns are matched before culprit patterns because an MPS teardown message can be followed by generic CUDA errors.

container/guard.py

```
#!/usr/bin/env python3
"""guard.py - error catcher that wraps the inference server inside the container.

Runs the real server as a child process, mirrors its stderr, and on exit writes
/run/gpushare/exit.json with the exit code and a classification of the cause.
The IWMR runtime reads that file when Docker reports the container as dead.

Usage (container ENTRYPOINT):  python3 /opt/gpushare/guard.py -- <server command...>
"""
import collections, json, os, re, signal, subprocess, sys, threading, time

STATUS_DIR = os.environ.get("GPUSHARE_STATUS_DIR", "/run/gpushare")

# Ordered: the first matching class wins.
PATTERNS = [
    ("gpu_fault_victim", r"MPS client terminated|MPS server.*(terminated|unavailable)|CUDA_ERROR_MPS"),
    ("gpu_fault_culprit", r"illegal memory access|illegal address|misaligned address|"
                          r"unspecified launch failure|illegal instruction|"
                          r"Memory access fault by GPU|hipErrorIllegalAddress"),
    ("device_error", r"GPU reset|launch timed out|hipErrorLaunchTimeOut|uncorrectable ECC|"
                     r"GPU is lost|fallen off the bus"),
    ("oom", r"out of memory|OutOfMemoryError|hipErrorOutOfMemory|\[hipmemcap\] denied"),
]


def classify(exit_code, tail):
    text = "\n".join(tail)
    for name, pat in PATTERNS:
```

```
            if re.search(pat, text, re.IGNORECASE):
                return name
        if exit_code == 0:
            return "ok"
        if exit_code in (-signal.SIGTERM, 128 + signal.SIGTERM):
            return "stopped"
        return "app_error"


    def main():
        cmd = sys.argv[sys.argv.index("--") + 1:] if "--" in sys.argv else sys.argv[1:]
        if not cmd:
            sys.exit("usage: guard.py -- <command>")

        os.makedirs(STATUS_DIR, exist_ok=True)
        tail = collections.deque(maxlen=200)          # last stderr lines for classification
        started = time.time()
        child = subprocess.Popen(cmd, stderr=subprocess.PIPE, text=True, bufsize=1)

        def forward(sig, _frame):                     # pass docker stop through to the server
            child.send_signal(sig)
        signal.signal(signal.SIGTERM, forward)
        signal.signal(signal.SIGINT, forward)

        def pump():
            for line in child.stderr:
                sys.stderr.write(line)
                tail.append(line.rstrip())
        t = threading.Thread(target=pump, daemon=True)
        t.start()

        code = child.wait()
        t.join(timeout=2)
        report = {
            "workload_id": os.environ.get("GPUSHARE_WORKLOAD_ID"),
            "exit_code": code,
            "class": classify(code, tail),
            "started": started,
            "ended": time.time(),
            "stderr_tail": list(tail)[-20:],
        }
        tmp = os.path.join(STATUS_DIR, "exit.json.tmp")
        with open(tmp, "w") as f:
            json.dump(report, f)
        os.replace(tmp, os.path.join(STATUS_DIR, "exit.json"))
        sys.exit(code if code >= 0 else 128 - code)


    if __name__ == "__main__":
        main()
```

Class	Typical evidence	IWMR action (§7.9)
oom	CUDA out of memory · OutOfMemoryError · hipErrorOutOfMemory · [hipmemcap] denied	Raise cap ×1.25, re-place
gpu_fault_culprit	illegal memory access · misaligned address · unspecified launch failure · Memory access fault by GPU; or kernel-log attribution	Drop: no restart, approval revoked, pending requests 503
gpu_fault_victim	MPS client terminated · CUDA_ERROR_MPS_*	Re-place immediately; router replays
device_error	GPU reset · launch timed out · uncorrectable ECC · fallen off the bus	Drain the GPU, re-place
app_error / unknown	anything else; no exit.json (SIGKILL)	Victim if the incident has a culprit, else back-off restart (max 5)
ok / stopped	exit 0 · SIGTERM	None

Table 4. Exit classes. MPS error names differ between CUDA versions; confirm them with test T9 and extend PATTERNS.

6.2 Recoverable OOM inside the server

An allocation over the ceiling raises an OOM exception but leaves the context usable. The server frees the allocator cache, halves its batch limit and returns 503 with Retry-After; the IWMR router waits and retries. Fatal faults are not caught: the process must exit for guard.py to report it.

container/oom_handling.py

```
"""oom_handling.py - recoverable-error handling inside the inference server.

An allocation that hits the MPS / hipmemcap limit raises an OOM error but leaves
the GPU context valid, so the server can shed load instead of dying. Fatal GPU
faults are NOT caught here: the context is unusable and guard.py reports them.
Works unchanged on PyTorch CUDA and PyTorch ROCm builds.
"""
import os, time
import torch
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()
model = None                                # loaded at startup from the read-only weights volume
OOM_EVENTS = 0
MAX_BATCH = int(os.environ.get("MAX_BATCH", "32"))


@app.on_event("startup")
def load():
    global model
    model = torch.load(os.environ["MODEL_PATH"], weights_only=False).eval().to("cuda")


@app.post("/infer")
def infer(payload: dict):
    global OOM_EVENTS, MAX_BATCH
    x = torch.tensor(payload["inputs"], device="cuda")
    try:
        with torch.inference_mode():
            y = model(x[:MAX_BATCH])
        return {"outputs": y.cpu().tolist()}
    except torch.OutOfMemoryError:
        OOM_EVENTS += 1
        torch.cuda.empty_cache()              # return cached blocks to the capped pool
        MAX_BATCH = max(1, MAX_BATCH // 2)    # degrade instead of crashing
        return JSONResponse({"error": "gpu_oom", "retry": True},
                            status_code=503, headers={"Retry-After": "1"})


@app.get("/health")
def health():
    # Polled by the IWMR runtime (readiness); oom_events shows memory pressure.
    return {"ok": True, "oom_events": OOM_EVENTS, "max_batch": MAX_BATCH,
            "mem_reserved": torch.cuda.memory_reserved(), "ts": time.time()}
```

container/Dockerfile.example

```
# Example inference image. The model server itself is unchanged; guard.py wraps it.
# NVIDIA: FROM nvcr.io/nvidia/pytorch:<tag>   AMD: FROM rocm/pytorch:<tag>
ARG BASE=nvcr.io/nvidia/pytorch:24.08-py3
FROM ${BASE}

RUN pip install --no-cache-dir fastapi uvicorn
COPY guard.py /opt/gpushare/guard.py
COPY oom_handling.py /app/server.py
# smoke_test.py is required by ci/validate_image.sh
COPY smoke_test.py /app/smoke_test.py

ENV GPUSHARE_STATUS_DIR=/run/gpushare
EXPOSE 8000
ENTRYPOINT ["python3", "/opt/gpushare/guard.py", "--"]
CMD ["uvicorn", "server:app", "--app-dir", "/app", "--host", "0.0.0.0", "--port", "8000"]
```

container/smoke_test_template.py

```
#!/usr/bin/env python3
"""smoke_test_template.py - copy into the image as /app/smoke_test.py.

The sanitizer only checks code paths that actually run, so exercise the shapes
production will see: smallest and largest batch, shortest and longest sequence,
and any special inputs (padding, empty prompts, max tokens).
"""
import os
import torch

MODEL_PATH = os.environ.get("MODEL_PATH", "/models/model.pt")
CASES = [(1, 16), (1, 4096), (32, 128), (64, 512)]    # (batch, sequence) - adapt per model

model = torch.load(MODEL_PATH, weights_only=False).eval().to("cuda")
with torch.inference_mode():
    for batch, seq in CASES:
        x = torch.randint(0, 1000, (batch, seq), device="cuda")
        model(x)
        torch.cuda.synchronize()          # surface asynchronous faults per case
        print(f"ok batch={batch} seq={seq}")
print("SMOKE TEST PASSED")
```

7. The IWMR runtime container

7.1 Image, privileges and start-up

The IWMR image contains no GPU libraries of its own; on NVIDIA the container toolkit injects nvidia-smi and libcuda (used once for the SM count), on AMD it only reads sysfs. Because the IWMR starts inference containers as *siblings* through the host's Docker socket, every path it hands to Docker must be a host path; the run script therefore mounts state, MPS and library directories at identical paths.

runtime/Dockerfile

```
# Inference Workload Management Runtime (IWMR) image. Vendor-neutral: the vendor
# is selected at run time with GPUSHARE_VENDOR. Build from the repository root:
#   docker build -f runtime/Dockerfile -t iwmr:latest .
FROM python:3.12-slim

RUN pip install --no-cache-dir fastapi uvicorn docker requests anyio
COPY runtime/ /opt/iwmr/
COPY tests/mps_sm_count.py /opt/iwmr/tools/mps_sm_count.py

WORKDIR /opt/iwmr
ENV PYTHONUNBUFFERED=1
EXPOSE 8080
CMD ["uvicorn", "runtime:app", "--host", "0.0.0.0", "--port", "8080"]
```

deploy/run_runtime.sh

```
#!/usr/bin/env bash
# run_runtime.sh - start the Inference Workload Management Runtime on this node.
#   sudo VENDOR=nvidia VALIDATION_GPU=3 ./deploy/run_runtime.sh
#   sudo VENDOR=amd ./deploy/run_runtime.sh
# Every path mounted below is identical on host and in the runtime, because the
# runtime starts sibling containers through the host's Docker daemon.
set -euo pipefail

VENDOR=${VENDOR:-nvidia}
IMAGE=${IMAGE:-iwmr:latest}
PORT=${PORT:-8080}

docker network inspect gpushare >/dev/null 2>&1 || docker network create gpushare
mkdir -p /var/lib/gpushare

COMMON=(
  -d --name iwmr --restart unless-stopped
  --network gpushare -p "$PORT:8080"
  -v /var/run/docker.sock:/var/run/docker.sock      # launch/stop inference containers
  -v /var/lib/gpushare:/var/lib/gpushare            # exit.json files, approved digests
  --pid host                                        # map faulting pids to containers
  --device /dev/kmsg --cap-add SYSLOG               # kernel log (Xid / amdgpu faults)
  -e GPUSHARE_VENDOR="$VENDOR"
  -e VALIDATION_GPU="${VALIDATION_GPU:-}"
```

```
)

if [ "$VENDOR" = nvidia ]; then
  docker run "${COMMON[@]}" \
    --gpus all --ipc host \
    -e NVIDIA_DRIVER_CAPABILITIES=compute,utility \
    -v /var/run/nvidia-mps:/var/run/nvidia-mps \
    -v "$(command -v nvidia-cuda-mps-control)":/usr/bin/nvidia-cuda-mps-control:ro \
    -e SANITIZER_DIR="${SANITIZER_DIR:-/usr/local/cuda/compute-sanitizer}" \
    "$IMAGE"
else
  docker run "${COMMON[@]}" \
    -v /opt/gpushare/lib:/opt/gpushare/lib:ro \
    "$IMAGE"
fi

sleep 3
curl -fsS "http://localhost:$PORT/health" && echo
```

Flag / mount	Needed by	Reason
/var/run/docker.sock	Launcher, exit watcher, validator	Start, stop and watch inference containers
/var/lib/gpushare (same path)	Launcher, exit watcher, validator	Per-workload exit.json directories; approved digests
--network gpushare	Router, readiness check	Reach containers by name (gs-<id>) without publishing ports
--pid host	Fault monitor	Map a faulting host pid to its container via /proc/<pid>/cgroup
--device /dev/kmsg, --cap-add SYSLOG	Fault monitor	Read kernel GPU fault records
--gpus all, --ipc host (NVIDIA)	Backend	nvidia-smi inventory; SM count as an MPS client
/var/run/nvidia-mps, mps-control binary (NVIDIA)	Backend health	Query each GPU's MPS daemon
/opt/gpushare/lib (AMD)	Launcher	Path of the memcap library handed to inference containers
VALIDATION_GPU	Validator, ledger	GPU excluded from sharing and used for memory-checker runs

Table 5. IWMR privileges and why each is needed.

Security: access to the Docker socket is equivalent to root on the host. The IWMR API must not be exposed beyond the trusted network until authentication is added (Section 10).

```
docker build -f runtime/Dockerfile -t iwmr:latest .          # from the gpushare/ root
sudo VENDOR=nvidia VALIDATION_GPU=3 ./deploy/run_runtime.sh
sudo VENDOR=amd ./deploy/run_runtime.sh
docker logs -f iwmr
curl -s localhost:8080/state | python3 -m json.tool
```

7.2 API

Method and path	Effect
POST /workloads	Register and place a workload (Table 2). 403 if a shared workload's digest is not approved; 503 if no GPU fits
GET /workloads, GET /workloads/{id}	Status (starting, ready, recovering, unhealthy, dropped, failed, stopped), GPU, allocation, restarts
DELETE /workloads/{id}	Graceful stop; the allocation is released when the container exits
POST /workloads/{id}/infer	Inference request through the router; optional X-Request-ID header as idempotency key
POST /images/validate, GET /images/validate/{job}	Run the memory checker on the validation GPU; job status and log tail
GET /state	Per GPU: health, share used, memory reserved, utilisation, unexpected use; in-flight requests; recent recovery events
POST /gpus/{g}/undrain	Return a drained GPU to service after inspection

Method and path	Effect
GET /health	Liveness of the IWMR

Table 6. IWMR REST API (interactive documentation at /docs).

```
curl -s -X POST localhost:8080/images/validate -d '{"image":"registry.local/infer/llama-8b:1.4"}' \
     -H 'Content-Type: application/json'                        # -> {"job": "1a2b3c4d"}
curl -s localhost:8080/images/validate/1a2b3c4d                 # status: approved | rejected
curl -s -X POST localhost:8080/workloads -H 'Content-Type: application/json' -d @deploy/example_request.json
curl -s -X POST localhost:8080/workloads/llm-a-001/infer -H 'Content-Type: application/json' -d '{"inputs":[[1,2,3]]}'
curl -s -X DELETE localhost:8080/workloads/llm-a-001
```

7.3 Core: registry, launcher and watchers (runtime.py)

place() runs admission, asks the backend for an allocation on each candidate GPU in best-fit order, launches the container and commits the share the backend granted. A readiness thread polls the container's /health. watch_exits() turns Docker 'die' events into recovery decisions; it ignores events from containers that have already been replaced and finalises deliberate stops. On start-up the IWMR removes any leftover workload containers, because its registry is not persistent.

runtime/runtime.py

```
#!/usr/bin/env python3
"""runtime.py - Inference Workload Management Runtime (IWMR).

One container per GPU node. It is the only management component: it exposes
the API, keeps the workload registry, admits and places workloads (best-fit),
launches inference containers with vendor-specific limits, routes inference
requests with an in-flight journal, watches exits and kernel-log faults,
recovers workloads, samples telemetry and validates images.

  docker run ... iwmr      (see deploy/run_runtime.sh for the required flags)
  API: http://<node>:8080/docs
"""
import json, os, threading, time
import anyio, docker, requests
from fastapi import FastAPI, HTTPException, Request

from backends.nvidia import NvidiaBackend
from backends.amd import AmdBackend
from ledger import Ledger
from router import Router
from recovery import Recovery
from faults import FaultMonitor
from telemetry import Telemetry
from validator import Validator

VENDOR = os.environ.get("GPUSHARE_VENDOR", "nvidia")
NETWORK = os.environ.get("GPUSHARE_NETWORK", "gpushare")
STATE = "/var/lib/gpushare"                  # same path on host and in the runtime
LABEL = "gpushare.workload_id"
VALIDATION_GPU = int(os.environ["VALIDATION_GPU"]) if os.environ.get("VALIDATION_GPU") else None
READY_TIMEOUT_S = int(os.environ.get("READY_TIMEOUT_S", "600"))
REQUIRED = ("workload_id", "image", "compute_share", "mem_bytes")

backend = NvidiaBackend() if VENDOR == "nvidia" else AmdBackend()
dock = docker.from_env()
ledger = Ledger()
registry = {}                                # wid -> {req, status, gpu, container, allocation}
lock = threading.RLock()
app = FastAPI(title="Inference Workload Management Runtime")

amd_minor = (backend.gpus[VALIDATION_GPU]["render_minor"]
             if VENDOR == "amd" and VALIDATION_GPU is not None else None)
validator = Validator(VENDOR, VALIDATION_GPU, amd_minor)
router = Router(registry, lambda wid: f"http://gs-{wid}:{registry[wid]['req'].get('port', 8000)}")
telemetry = Telemetry(VENDOR, backend)


# ------------------------------------------------------------------ placement and launch
def launch(req, alloc):
    wid = req["workload_id"]
    status_dir = f"{STATE}/{wid}"
    os.makedirs(status_dir, exist_ok=True)
    try:
        os.remove(f"{status_dir}/exit.json")
```

```
    except FileNotFoundError:
        pass
    try:                                                # leftover from a previous run
        dock.containers.get(f"gs-{wid}").remove(force=True)
    except docker.errors.NotFound:
        pass
    kw = backend.container_kwargs(req, alloc)
    kw["environment"].update(req.get("env", {}), GPUSHARE_WORKLOAD_ID=wid)
    kw["volumes"][status_dir] = {"bind": "/run/gpushare", "mode": "rw"}
    for host, cont in req.get("volumes", {}).items():    # e.g. read-only model weights
        kw["volumes"][host] = {"bind": cont, "mode": "ro"}
    c = dock.containers.run(req["image"], command=req.get("command"), detach=True,
                            name=f"gs-{wid}", network=NETWORK,
                            labels={LABEL: wid, "gpushare.gpu": str(alloc.gpu)},
                            restart_policy={"Name": "no"}, **kw)
    return c.id


def place(req):
    """Admission + best-fit placement + launch. Raises RuntimeError if nothing fits."""
    wid = req["workload_id"]
    with lock:
        for g in ledger.candidates(req):
            try:
                alloc = backend.allocate(req, g)         # AMD may refuse: CU fragmentation
            except ValueError:
                continue
            try:
                cid = launch(req, alloc)
            except docker.errors.APIError as e:
                backend.release(wid)
                raise RuntimeError(f"docker run failed: {e}")
            ledger.commit(req, g, alloc.compute_share)
            rec = registry.setdefault(wid, {})
            rec.update(req=req, status="starting", gpu=g, container=cid,
                       allocation=alloc.__dict__, started=time.time())
            threading.Thread(target=wait_ready, args=(wid, cid), daemon=True).start()
            return rec
        raise RuntimeError(f"no feasible GPU: {ledger.explain(req)}")


def wait_ready(wid, cid):
    t0 = time.time()
    while time.time() - t0 < READY_TIMEOUT_S:
        rec = registry.get(wid)
        if not rec or rec.get("container") != cid or rec["status"] != "starting":
            return
        try:
            if requests.get(router.host(wid) + "/health", timeout=2).ok:
                rec["status"] = "ready"
                return
        except requests.RequestException:
            pass
        time.sleep(2)
    if registry.get(wid, {}).get("container") == cid:
        registry[wid]["status"] = "unhealthy"


# ---------------------------------------------------------------------- background watchers
def watch_exits():
    for ev in dock.events(decode=True, filters={"type": "container", "event": "die", "label": LABEL}):
        a = ev["Actor"]["Attributes"]
        wid, gpu = a.get(LABEL), int(a.get("gpushare.gpu", -1))
        try:
            report = json.load(open(f"{STATE}/{wid}/exit.json"))
        except (OSError, ValueError):
            report = {"class": "unknown", "exit_code": int(a.get("exitCode", -1))}
        with lock:
            backend.release(wid)
            ledger.release(wid)
            rec = registry.get(wid)
        try:
            dock.containers.get(ev["id"]).remove()
        except docker.errors.APIError:
            pass
        if rec is None or rec.get("container") != ev["id"]:
            continue
        if rec["status"] == "stopping":
            registry.pop(wid, None)
            continue
        threading.Thread(target=recovery.on_exit, args=(wid, gpu, report), daemon=True).start()

def on_kernel_fault(evt):
```

```
    if evt["device_level"]:
        g = ledger.gpu_of_pci(evt["pci"])
        if g is not None:
            ledger.gpus[g].healthy = False              # drain; containers exit on their own
        recovery._log(None, "device_error", f"kernel: {evt['raw'][:120]}")
        return
    if evt.get("container"):
        try:
            wid = dock.containers.get(evt["container"]).labels.get(LABEL)
        except docker.errors.NotFound:
            wid = None
        if wid:
            recovery.mark_kernel_culprit(wid)


def watch_health(period=10):
    while True:
        for g in list(ledger.gpus):
            h = backend.health(g)
            if not h.get("ok") and ledger.gpus[g].healthy:
                ledger.gpus[g].healthy = False
                recovery._log(None, "backend", f"gpu {g} unhealthy: {h}")
        time.sleep(period)


recovery = Recovery(ledger, registry, place, validator)


@app.on_event("startup")
def startup():
    os.makedirs(STATE, exist_ok=True)
    for g in backend.inventory():
        if g["gpu"] != VALIDATION_GPU:
            ledger.add_gpu(g["gpu"], g["compute_units"], g["mem_bytes"], g.get("pci", ""))
    for c in dock.containers.list(all=True, filters={"label": LABEL}):
        c.remove(force=True)                            # registry is not persistent: start clean
    for t in (watch_exits, watch_health):
        threading.Thread(target=t, daemon=True).start()
    FaultMonitor(on_kernel_fault).start()
    telemetry.start()


# ------------------------------------------------------------------ API
@app.post("/workloads")
def submit(req: dict):
    missing = [k for k in REQUIRED if k not in req]
    if missing:
        raise HTTPException(422, f"missing fields: {missing}")
    if req["workload_id"] in registry and registry[req["workload_id"]]["status"] not in ("dropped", "failed",
  "stopped"):
        raise HTTPException(409, "workload_id already exists")
    if req.get("isolation", "shared") == "shared" and not validator.approved(req.get("image_digest")):
        raise HTTPException(403, "image digest not validated; validate it or use isolation=exclusive")
    try:
        return place(req)
    except RuntimeError as e:
        raise HTTPException(503, str(e))


@app.get("/workloads")
def list_workloads():
    return {w: {k: r.get(k) for k in ("status", "gpu", "allocation", "restarts")}
            for w, r in registry.items()}


@app.get("/workloads/{wid}")
def get_workload(wid: str):
    if wid not in registry:
        raise HTTPException(404)
    return registry[wid]


@app.delete("/workloads/{wid}")
def stop(wid: str):
    rec = registry.get(wid)
    if not rec:
        raise HTTPException(404)
    rec["status"] = "stopping"
    try:
        dock.containers.get(f"gs-{wid}").stop(timeout=20)
    except docker.errors.NotFound:
        registry.pop(wid, None)
    return {"stopping": wid}
```

```
@app.post("/workloads/{wid}/infer")
async def infer(wid: str, request: Request):
    if wid not in registry:
        raise HTTPException(404)
    payload = await request.json()
    code, body = await anyio.to_thread.run_sync(
        router.forward, wid, payload, request.headers.get("X-Request-ID"))
    if code >= 400:
        raise HTTPException(code, body)
    return body


@app.post("/images/validate")
def validate(body: dict):
    try:
        return {"job": validator.submit(body["image"])}
    except RuntimeError as e:
        raise HTTPException(409, str(e))


@app.get("/images/validate/{job}")
def validation_job(job: str):
    return validator.jobs.get(job) or {}


@app.get("/state")
def state():
    gpus = {}
    for g, s in ledger.gpus.items():
        share, mem = s.used()
        t = telemetry.latest.get(g, {})
        gpus[g] = {"healthy": s.healthy, "share_used": round(share, 3),
                   "mem_reserved": mem, "workloads": list(s.workloads), **t,
                   "unexpected_use": t.get("mem_used", 0) > mem + 2**30}
    return {"vendor": VENDOR, "gpus": gpus, "validation_gpu": VALIDATION_GPU,
            "inflight_requests": router.inflight(), "recent_events": recovery.events[-20:]}


@app.post("/gpus/{g}/undrain")
def undrain(g: int):
    ledger.gpus[g].healthy = True
    return {"gpu": g, "healthy": True}


@app.get("/health")
def health():
    return {"ok": True, "vendor": VENDOR, "workloads": len(registry)}
```

7.4 Vendor backends

Both vendors implement the same interface. The NVIDIA backend reads the inventory once (including the full SM count, measured as an MPS client at 100 %) and points each container at its GPU's MPS pipe directory. The AMD backend reads the KFD topology, tracks free CUs per GPU and releases a range when its workload exits.

`runtime/backends/base.py`

```
"""Vendor-neutral backend interface used by the runtime (IWMR)."""
from dataclasses import dataclass, field


@dataclass
class Allocation:
    workload_id: str
    gpu: int                      # host GPU index
    compute_share: float          # granted share, 0..1 (after rounding)
    mem_bytes: int                # granted device-memory cap
    detail: dict = field(default_factory=dict)   # vendor specifics (e.g. CU set)


class Backend:
    vendor = "base"

    def inventory(self) -> list:
        """[{gpu, uuid, name, compute_units, mem_bytes, fault_domain}, ...]"""
        raise NotImplementedError

    def allocate(self, req: dict, gpu: int) -> Allocation:
        """Reserve resources on `gpu` for request `req`; raise ValueError if impossible."""
        raise NotImplementedError
```

```
    def release(self, workload_id: str) -> None:
        raise NotImplementedError

    def container_kwargs(self, req: dict, alloc: Allocation) -> dict:
        """docker-py run() kwargs that enforce the allocation inside the container."""
        raise NotImplementedError

    def health(self, gpu: int) -> dict:
        """Backend health for one GPU (e.g. MPS daemon reachable)."""
        return {"ok": True}
```

runtime/backends/nvidia.py

```
"""NVIDIA backend: MPS active-thread percentage + MPS pinned device-memory limit."""
import math, os, subprocess
import docker
from .base import Allocation, Backend

MPS_ROOT = "/var/run/nvidia-mps"
SM_COUNT_HELPER = os.environ.get("SM_COUNT_HELPER", "/opt/iwmr/tools/mps_sm_count.py")


def _pipe_dir(gpu):
    return f"{MPS_ROOT}/gpu{gpu}"


class NvidiaBackend(Backend):
    vendor = "nvidia"
    MAX_CLIENTS = 48            # per MPS server on Volta+; check your CUDA version

    def __init__(self):
        self.allocs = {}        # workload_id -> Allocation
        self._inv = None        # static topology, discovered once

    def inventory(self):
        if self._inv is None:
            self._inv = self._discover()
        return [dict(g) for g in self._inv]

    def _discover(self):
        out = subprocess.run(
            ["nvidia-smi", "--query-gpu=index,uuid,name,memory.total,pci.bus_id",
             "--format=csv,noheader,nounits"],
            capture_output=True, text=True, check=True).stdout
        gpus = []
        for line in out.strip().splitlines():
            idx, uuid, name, mem_mib, pci = [s.strip() for s in line.split(",")]
            gpus.append({"gpu": int(idx), "uuid": uuid, "name": name,
                         "compute_units": self._sm_count(int(idx)),
                         "mem_bytes": int(mem_mib) * 2**20,
                         "fault_domain": f"mps-gpu{idx}",
                         "pci": pci.lower()[-12:]})   # "0000:3b:00.0" form
        return gpus

    def _sm_count(self, gpu):
        # Full SM count: query as an MPS client with a 100 % limit.
        env = dict(os.environ, CUDA_VISIBLE_DEVICES=str(gpu),
                   CUDA_MPS_PIPE_DIRECTORY=_pipe_dir(gpu),
                   CUDA_MPS_ACTIVE_THREAD_PERCENTAGE="100")
        r = subprocess.run(["python3", SM_COUNT_HELPER], env=env,
                           capture_output=True, text=True)
        return int(r.stdout.strip() or 0)

    def allocate(self, req, gpu):
        on_gpu = [a for a in self.allocs.values() if a.gpu == gpu]
        if len(on_gpu) >= self.MAX_CLIENTS:
            raise ValueError("MPS client limit reached")
        pct = max(1, min(100, math.ceil(req["compute_share"] * 100)))
        a = Allocation(req["workload_id"], gpu, pct / 100, int(req["mem_bytes"]),
                       {"active_thread_percentage": pct})
        self.allocs[a.workload_id] = a
        return a

    def release(self, workload_id):
        self.allocs.pop(workload_id, None)

    def container_kwargs(self, req, alloc):
        pipe = _pipe_dir(alloc.gpu)
        mem_mib = alloc.mem_bytes // 2**20
        return {
            "device_requests": [docker.types.DeviceRequest(
                device_ids=[str(alloc.gpu)], capabilities=[["gpu"]])],
            "ipc_mode": "host",                             # required for MPS clients
```

```
            "volumes": {pipe: {"bind": pipe, "mode": "rw"}},
            "environment": {
                "CUDA_MPS_PIPE_DIRECTORY": pipe,
                "CUDA_MPS_ACTIVE_THREAD_PERCENTAGE": str(alloc.detail["active_thread_percentage"]),
                # device index as seen INSIDE the container (one GPU -> 0)
                "CUDA_MPS_PINNED_DEVICE_MEM_LIMIT": f"0={mem_mib}M",
            },
        }

    def health(self, gpu):
        if not os.path.isdir(_pipe_dir(gpu)):
            return {"ok": False, "reason": "no MPS daemon for this GPU"}
        env = dict(os.environ, CUDA_MPS_PIPE_DIRECTORY=_pipe_dir(gpu))
        r = subprocess.run(["nvidia-cuda-mps-control"], input="get_server_list\n",
                           env=env, capture_output=True, text=True, timeout=5)
        return {"ok": r.returncode == 0, "mps_servers": r.stdout.split()}
```

runtime/backends/amd.py

```
"""AMD backend: disjoint CU sets via HSA_CU_MASK + hipMalloc interposer for memory."""
import glob, grp, math, os
from .base import Allocation, Backend


def _has_group(name):
    try:
        grp.getgrnam(name)
        return True
    except KeyError:
        return False

KFD_NODES = "/sys/class/kfd/kfd/topology/nodes"
MEMCAP_LIB = "/opt/gpushare/lib/libhipmemcap.so"


def _props(path):
    out = {}
    for line in open(path):
        k, _, v = line.partition(" ")
        if v.strip().lstrip("-").isdigit():
            out[k] = int(v)
    return out


class AmdBackend(Backend):
    vendor = "amd"

    def __init__(self):
        self.allocs = {}                # workload_id -> Allocation
        self.gpus = self._discover()    # gpu index -> topology info
        self.free = {g: [True] * info["compute_units"] for g, info in self.gpus.items()}

    def _discover(self):
        gpus, idx = {}, 0
        for node in sorted(glob.glob(f"{KFD_NODES}/*"), key=lambda p: int(p.rsplit("/", 1)[1])):
            p = _props(f"{node}/properties")
            if p.get("simd_count", 0) == 0:
                continue                                # CPU node
            minor = p["drm_render_minor"]
            try:
                vram = int(open(f"/sys/class/drm/renderD{minor}/device/mem_info_vram_total").read())
            except OSError:
                vram = 0
            pci = os.path.basename(os.path.realpath(f"/sys/class/drm/renderD{minor}/device"))
            gpus[idx] = {"render_minor": minor, "pci": pci.lower()[-12:],
                         "compute_units": p["simd_count"] // p["simd_per_cu"],
                         "num_xcc": p.get("num_xcc", 1), "mem_bytes": vram}
            idx += 1
        return gpus

    def inventory(self):
        return [{"gpu": g, "uuid": f"renderD{i['render_minor']}", "name": "amd-gpu",
                 "compute_units": i["compute_units"], "mem_bytes": i["mem_bytes"],
                 "free_compute_units": sum(self.free[g]),
                 "fault_domain": f"amd-gpu{g}", "pci": i["pci"]} for g, i in self.gpus.items()]

    def _find_range(self, gpu, n):
        """First-fit contiguous CU range, preferring one that stays inside one XCC."""
        free, total = self.free[gpu], len(self.free[gpu])
        per_xcc = total // self.gpus[gpu]["num_xcc"]
        starts = range(0, total - n + 1)
        if n <= per_xcc:   # try ranges that do not cross an XCC boundary first
            starts = sorted(starts, key=lambda s: (s // per_xcc != (s + n - 1) // per_xcc, s))
```

```
        for s in starts:
            if all(free[s:s + n]):
                return s
        raise ValueError("no contiguous CU range free (fragmentation)")

    def allocate(self, req, gpu):
        total = self.gpus[gpu]["compute_units"]
        n = max(2, math.ceil(req["compute_share"] * total))
        n += n % 2                                          # keep CU pairs together
        start = self._find_range(gpu, n)
        for c in range(start, start + n):
            self.free[gpu][c] = False
        a = Allocation(req["workload_id"], gpu, n / total, int(req["mem_bytes"]),
                       {"cu_first": start, "cu_last": start + n - 1})
        self.allocs[a.workload_id] = a
        return a

    def release(self, workload_id):
        a = self.allocs.pop(workload_id, None)
        if a:
            for c in range(a.detail["cu_first"], a.detail["cu_last"] + 1):
                self.free[a.gpu][c] = True

    def container_kwargs(self, req, alloc):
        minor = self.gpus[alloc.gpu]["render_minor"]
        return {
            "devices": ["/dev/kfd:/dev/kfd", f"/dev/dri/renderD{minor}:/dev/dri/renderD{minor}"],
            # numeric host GIDs: group names may not exist inside the image
            "group_add": [str(grp.getgrnam(g).gr_gid) for g in ("video", "render") if _has_group(g)],
            "volumes": {MEMCAP_LIB: {"bind": MEMCAP_LIB, "mode": "ro"}},
            "environment": {
                # one GPU visible inside the container -> device index 0
                "HSA_CU_MASK": f"0:{alloc.detail['cu_first']}-{alloc.detail['cu_last']}",
                "HIPMEM_LIMIT_BYTES": str(alloc.mem_bytes),
                "LD_PRELOAD": MEMCAP_LIB,
            },
        }
```

7.5 Admission and best-fit placement

The ledger turns ceilings into effective reservations: a GPU is feasible only if the sums of shares and of memory caps plus context overhead stay within capacity, the client limit is respected and exclusivity holds. Among feasible GPUs, best-fit takes the one with the least compute share left after placement (ties: lowest index). This packs workloads and keeps whole GPUs free for large or exclusive ones; it is deterministic. `ledger.explain(req)` gives the reason each GPU was rejected and is returned with a 503.

`runtime/ledger.py`

```
"""ledger.py - admission and deterministic placement for the GPUs of this node.

MPS percentages, CU masks and memory caps are ceilings, not reservations. The
ledger admits a workload on a GPU only if the sum of granted compute shares
stays <= 1 and the sum of memory caps plus per-client context overhead stays
within device memory. Under that rule the ceilings behave as reservations.

Placement is plain best-fit: among feasible GPUs, take the one that is left
with the least free compute share (ties: lowest index). No learning, no scoring.
"""
import os
from dataclasses import dataclass, field

CTX_OVERHEAD = int(os.environ.get("CTX_OVERHEAD_MB", "512")) * 2**20   # measure per GPU model
COMPUTE_HEADROOM = float(os.environ.get("COMPUTE_HEADROOM", "0.0"))     # e.g. 0.1 kept free
MAX_CLIENTS = int(os.environ.get("MAX_CLIENTS_PER_GPU", "48"))


@dataclass
class Gpu:
    index: int
    compute_units: int
    mem_bytes: int
    pci: str = ""
    healthy: bool = True
    workloads: dict = field(default_factory=dict)  # wid -> (share, mem, exclusive)

    def used(self):
        return (sum(w[0] for w in self.workloads.values()),
```

```
                sum(w[1] + CTX_OVERHEAD for w in self.workloads.values()))

class Ledger:
    def __init__(self):
        self.gpus = {}

    def add_gpu(self, index, compute_units, mem_bytes, pci=""):
        self.gpus[index] = Gpu(index, compute_units, mem_bytes, pci)

    def fits(self, req, g):
        gpu = self.gpus[g]
        if not gpu.healthy:
            return False, "gpu unhealthy (drained)"
        if any(w[2] for w in gpu.workloads.values()):
            return False, "gpu held by an exclusive workload"
        if req.get("isolation") == "exclusive" and gpu.workloads:
            return False, "exclusive placement needs an empty gpu"
        if len(gpu.workloads) >= MAX_CLIENTS:
            return False, "client limit"
        share, mem = gpu.used()
        if share + req["compute_share"] > 1.0 - COMPUTE_HEADROOM + 1e-9:
            return False, "compute share exhausted"
        if mem + req["mem_bytes"] + CTX_OVERHEAD > gpu.mem_bytes:
            return False, "device memory exhausted"
        return True, "ok"

    def candidates(self, req):
        """Feasible GPUs in best-fit order."""
        ok = [g for g in sorted(self.gpus) if self.fits(req, g)[0]]
        return sorted(ok, key=lambda g: (1.0 - self.gpus[g].used()[0] - req["compute_share"], g))

    def explain(self, req):
        return {g: self.fits(req, g)[1] for g in sorted(self.gpus)}

    def commit(self, req, g, granted_share):
        self.gpus[g].workloads[req["workload_id"]] = (
            granted_share, req["mem_bytes"], req.get("isolation") == "exclusive")

    def release(self, wid):
        for gpu in self.gpus.values():
            gpu.workloads.pop(wid, None)

    def gpu_of_pci(self, pci_fragment):
        for g in self.gpus.values():
            if g.pci and pci_fragment and pci_fragment in g.pci:
                return g.index
        return None
```

7.6 Request router and journal

Every inference request is journaled until its response returns. If the container is gone, the router waits (up to 180 s) for the workload to be ready again and replays the request; OOM back-pressure (503 with Retry-After) is retried after the advised delay. Requests to a dropped or failed workload end with 503. The journal is in memory: if the IWMR itself restarts, clients must retry. Servers should treat X-Request-ID as an idempotency key, since a request may be executed twice if a container dies after computing but before answering.

runtime/router.py

```
"""router.py - inference request router with an in-flight journal.

Clients send inference requests to the runtime, never to containers directly.
Each request is journaled until its response returns. If the target container
dies (fault, OOM exit, neighbour's MPS teardown), the request stays in the
journal, the router waits for the recovery controller to bring the workload
back, and replays it. Clients see added latency instead of an error.
"""
import threading, time, uuid
import requests

RECOVERY_WAIT_S = 180          # how long a request may wait for its workload to return
MAX_ATTEMPTS = 3
REQUEST_TIMEOUT_S = 120


class Router:
    def __init__(self, registry, network_host):
```

```
        self.registry = registry          # wid -> record with "status", "req"
        self.host = network_host          # callable(wid) -> "http://gs-<wid>:<port>"
        self.journal = {}                 # request_id -> {wid, ts, attempts}
        self.lock = threading.Lock()

    def inflight(self):
        with self.lock:
            out = {}
            for r in self.journal.values():
                out[r["wid"]] = out.get(r["wid"], 0) + 1
            return out

    def _wait_ready(self, wid):
        t0 = time.time()
        while time.time() - t0 < RECOVERY_WAIT_S:
            rec = self.registry.get(wid)
            if rec is None or rec["status"] in ("failed", "stopping", "dropped"):
                return False
            if rec["status"] == "ready":
                return True
            time.sleep(0.5)
        return False

    def forward(self, wid, payload, request_id=None):
        rid = request_id or str(uuid.uuid4())
        with self.lock:
            self.journal[rid] = {"wid": wid, "ts": time.time(), "attempts": 0}
        try:
            for attempt in range(1, MAX_ATTEMPTS + 1):
                self.journal[rid]["attempts"] = attempt
                if not self._wait_ready(wid):
                    return 503, {"error": "workload unavailable", "request_id": rid}
                path = self.registry[wid]["req"].get("infer_path", "/infer")
                try:
                    r = requests.post(self.host(wid) + path, json=payload, timeout=REQUEST_TIMEOUT_S,
                                      headers={"X-Request-ID": rid})
                except requests.ConnectionError:
                    time.sleep(1)           # container gone: the exit watcher starts recovery
                    continue
                if r.status_code == 503 and r.headers.get("Retry-After"):
                    time.sleep(float(r.headers["Retry-After"]))   # OOM back-pressure
                    continue
                return r.status_code, r.json()
            return 503, {"error": "retries exhausted", "request_id": rid}
        finally:
            with self.lock:
                self.journal.pop(rid, None)
```

7.7 Exit and fault monitor

The primary signal is exit.json from guard.py, read in watch_exits(). faults.py adds kernel-log attribution: on AMD the amdgpu page-fault record names the faulting pid, which is mapped to a container through its cgroup and marks that workload as culprit. On NVIDIA the Xid record may name the MPS server rather than the client, so it rarely maps to a container; device-level Xids (48, 74, 79, 95) and AMD resets drain the GPU found by PCI address.

runtime/faults.py

```
"""faults.py - kernel-log GPU fault monitor (thread inside the runtime).

Reads /dev/kmsg (runtime started with --device /dev/kmsg --cap-add SYSLOG) and
attributes GPU fault records to containers through /proc/<pid>/cgroup (needs
--pid host). NVIDIA Xid records may name the MPS server rather than the client,
so they corroborate guard.py; amdgpu page-fault records name the process.
"""
import re, threading

XID = re.compile(r"NVRM: Xid \(PCI:(?P<pci>[0-9a-fA-F:.]+)\): (?P<xid>\d+)"
                 r"(?:, pid=(?P<pid>\d+|'<unknown>'), name=(?P<name>[^,]+))?")
AMD_PF = re.compile(r"amdgpu (?P<pci>[0-9a-fA-F:.]+): \[\w+\] (?:\w+ )?page fault.*?"
                    r"process (?P<name>\S+) pid (?P<pid>\d+)")
AMD_RESET = re.compile(r"amdgpu (?P<pci>[0-9a-fA-F:.]+): .*GPU reset")
FATAL_XIDS = {13, 31, 43, 45, 48, 62, 63, 64, 74, 79, 94, 95}
DEVICE_XIDS = {48, 74, 79, 95}


def bus_dev(pci):
    """'0000:3b:00.0' / 'PCI:0000:3b:00' -> '3b:00' (comparable across sources)."""
    m = re.search(r"([0-9a-fA-F]{2}):([0-9a-fA-F]{2})(?:\.\d)?$", pci or "")
```

```
    return f"{m.group(1)}:{m.group(2)}".lower() if m else ""


def container_of(pid):
    try:
        m = re.search(r"([0-9a-f]{64})", open(f"/proc/{pid}/cgroup").read())
        return m.group(1) if m else None
    except OSError:
        return None


def parse(line):
    if m := XID.search(line):
        xid = int(m["xid"])
        if xid not in FATAL_XIDS:
            return None
        pid = m["pid"] if m["pid"] and m["pid"].isdigit() else None
        return {"source": "nvidia_xid", "xid": xid, "pci": bus_dev(m["pci"]), "pid": pid,
                "device_level": xid in DEVICE_XIDS}
    if m := AMD_PF.search(line):
        return {"source": "amdgpu_pagefault", "pci": bus_dev(m["pci"]), "pid": m["pid"],
                "device_level": False}
    if m := AMD_RESET.search(line):
        return {"source": "amdgpu_reset", "pci": bus_dev(m["pci"]), "pid": None,
                "device_level": True}
    return None


class FaultMonitor(threading.Thread):
    def __init__(self, on_fault, path="/dev/kmsg"):
        super().__init__(daemon=True)
        self.on_fault, self.path = on_fault, path

    def run(self):
        try:
            f = open(self.path, "r", errors="replace")
            f.seek(0, 2)                                    # only new records
        except OSError as e:
            print(f"[faults] kernel log unavailable ({e}); relying on guard.py only")
            return
        while True:
            try:
                line = f.readline()
            except OSError:                                 # kmsg overrun: skip ahead
                continue
            evt = parse(line.split(";", 1)[-1])
            if evt:
                evt["container"] = container_of(evt["pid"]) if evt["pid"] else None
                evt["raw"] = line.strip()[:400]
                self.on_fault(evt)
```

7.8 Telemetry

Utilisation and used memory per GPU are sampled every 5 s. Under MPS these are GPU-wide values. They are reported in /state; memory use more than 1 GiB above the ledger's reservations is flagged as *unexpected_use*, indicating GPU work started outside the IWMR. For bandwidth-level counters, DCGM (NVIDIA) or the AMD device metrics exporter can be run alongside, but the IWMR does not depend on them.

`runtime/telemetry.py`

```
"""telemetry.py - periodic device utilisation and memory sampling.

Values are GPU-wide (MPS does not report per client). They are exposed in
/state for operators and used by one admission guard: a GPU whose measured
memory use already exceeds the ledger's expectation is flagged, because
something outside the runtime is using it.
"""
import glob, subprocess, threading, time


def sample_nvidia():
    out = subprocess.run(["nvidia-smi", "--query-gpu=index,utilization.gpu,memory.used",
                          "--format=csv,noheader,nounits"], capture_output=True, text=True).stdout
    res = {}
    for line in out.strip().splitlines():
        i, util, mem = [x.strip() for x in line.split(",")]
        res[int(i)] = {"util_pct": float(util), "mem_used": int(mem) * 2**20}
    return res
```

```
def sample_amd(gpus):
    res = {}
    for g, info in gpus.items():
        dev = f"/sys/class/drm/renderD{info['render_minor']}/device"
        try:
            res[g] = {"util_pct": float(open(f"{dev}/gpu_busy_percent").read()),
                      "mem_used": int(open(f"{dev}/mem_info_vram_used").read())}
        except OSError:
            pass
    return res


class Telemetry(threading.Thread):
    def __init__(self, vendor, backend, period=5):
        super().__init__(daemon=True)
        self.vendor, self.backend, self.period = vendor, backend, period
        self.latest, self.ts = {}, 0

    def run(self):
        while True:
            try:
                self.latest = (sample_nvidia() if self.vendor == "nvidia"
                               else sample_amd(self.backend.gpus))
                self.ts = time.time()
            except Exception as e:                          # never kill the runtime
                print(f"[telemetry] sample failed: {e}")
            time.sleep(self.period)
```

7.9 Recovery controller

Exits on one GPU within 15 s form one incident. Rules are fixed. The **culprit is dropped**: it is not restarted, its image digest is removed from the approved list (it may only share a GPU again after re-validation, or run with isolation=exclusive), and its pending requests end with 503. Victims, and app_error exits inside an incident that has a culprit, are re-placed at once and the router replays their requests. OOM exits are re-placed with a 25 % larger cap (bounded by GPU memory); device errors drain the GPU. Other failures restart with exponential back-off, at most five times.

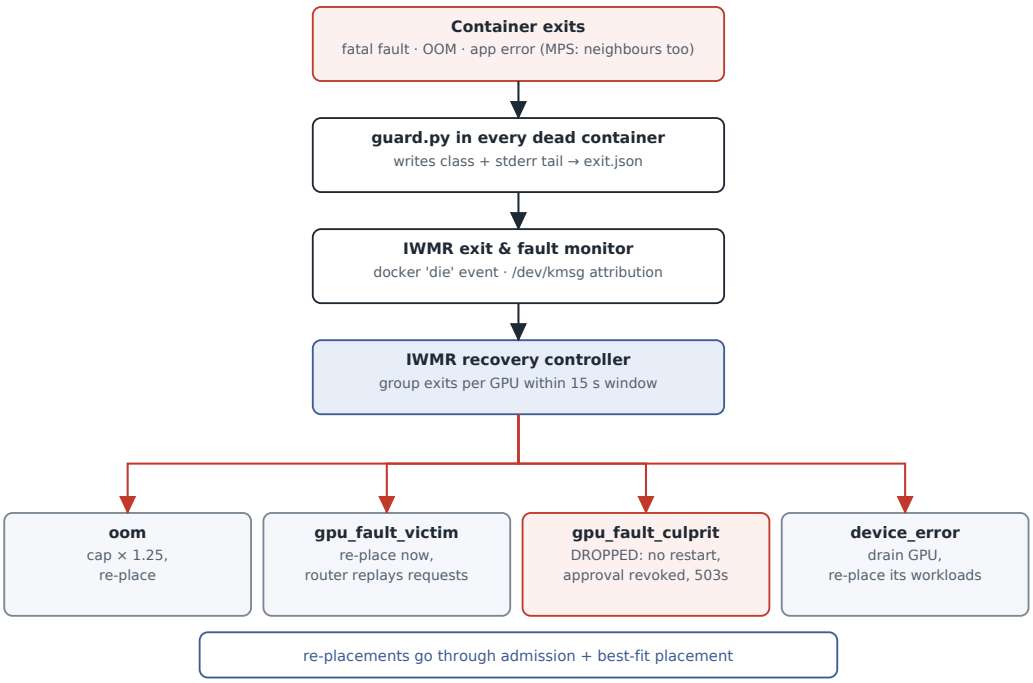


Figure 2. Exit classification and recovery.

runtime/recovery.py

```
"""recovery.py - decide what happens after a workload container exits.

Rules are fixed (no learning):
  ok/stopped        -> nothing
  oom               -> raise memory cap x1.25 (up to the GPU size), re-place
  gpu_fault_victim  -> re-place at once (router replays its in-flight requests)
  gpu_fault_culprit -> DROPPED: container not restarted, image approval revoked
                       (it can only return after re-validation); its in-flight
                       requests are failed back to the clients
  device_error      -> drain the GPU, re-place
  app_error/unknown -> victim if the same GPU had a culprit in the incident
                       window, otherwise restart with exponential back-off
Kernel-log attribution (faults.py) can upgrade an exit to culprit.
"""
import threading, time
from collections import defaultdict

WINDOW_S = 15
OOM_GROWTH = 1.25
MAX_RESTARTS = 5


class Recovery:
    def __init__(self, ledger, registry, place_fn, validator):
        self.ledger, self.registry, self.place = ledger, registry, place_fn
        self.validator = validator
        self.recent = defaultdict(list)       # gpu -> [(ts, wid, class)]
        self.kernel_culprits = {}             # wid -> ts (from faults.py)
        self.events = []                      # audit trail, exposed in /state
        self.lock = threading.Lock()

    def mark_kernel_culprit(self, wid):
        self.kernel_culprits[wid] = time.time()

    def on_exit(self, wid, gpu, report):
        rec = self.registry.get(wid)
        if rec is None:
            return
        req, now = rec["req"], time.time()
        cls = report.get("class", "unknown")
        if now - self.kernel_culprits.pop(wid, 0) < WINDOW_S and cls != "oom":
            cls = "gpu_fault_culprit"                    # kernel log named this process
        with self.lock:
            self.recent[gpu] = [r for r in self.recent[gpu] if now - r[0] < WINDOW_S]
            self.recent[gpu].append((now, wid, cls))
            incident = {c for _, _, c in self.recent[gpu]}

        if cls in ("ok", "stopped"):
            rec["status"] = "stopped"
            return self._log(wid, cls, "no action")
        if cls == "oom":
            cap = self.ledger.gpus[gpu].mem_bytes
            req["mem_bytes"] = min(int(req["mem_bytes"] * OOM_GROWTH), cap)
            return self._restart(wid, cls, "memory cap raised")
        if cls == "gpu_fault_culprit":
            self.validator.revoke(req.get("image_digest", req["image"]))
            rec["status"] = "dropped"                    # router fails its pending requests
            return self._log(wid, cls, "culprit dropped; image approval revoked")
        if cls == "gpu_fault_victim" or "gpu_fault_culprit" in incident:
            return self._restart(wid, cls, "victim of a co-located fault")
        if cls == "device_error":
            self.ledger.gpus[gpu].healthy = False
            return self._restart(wid, cls, f"gpu {gpu} drained")
        return self._restart(wid, cls, "application error", backoff=True)

    def _restart(self, wid, cls, reason, backoff=False):
        rec = self.registry[wid]
        rec["status"] = "recovering"
        rec["restarts"] = rec.get("restarts", 0) + 1
        self._log(wid, cls, reason)
        if rec["restarts"] > MAX_RESTARTS:
            rec["status"] = "failed"
            return self._log(wid, cls, "restart limit reached")
        delay = min(30, 2 ** rec["restarts"]) if backoff else 0
        last = ""
        for attempt in range(6):                         # wait for capacity (e.g. after a drain)
            time.sleep(delay)
            try:
                self.place(rec["req"])
                return self._log(wid, cls, "re-placed")
            except RuntimeError as e:
                delay = 10
```

```
                last = str(e)
        rec["status"] = "failed"
        return self._log(wid, cls, f"no capacity: {last}")

    def _log(self, wid, cls, action):
        self.events.append({"ts": time.time(), "workload_id": wid, "class": cls, "action": action})
        del self.events[:-200]
        return action
```

7.10 Image validator

Memory checkers slow execution by one to two orders of magnitude and examine only code paths that run, so they gate admission rather than protect production. The validator runs the image's smoke test as a temporary container on the validation GPU. On NVIDIA, compute-sanitizer from the host CUDA toolkit is mounted into the unmodified image (the image's glibc must be compatible; otherwise build a CI variant containing it). On AMD no checker works on an unmodified binary: submit an image variant whose HIP code is compiled with `-fsanitize=address -shared-libsan -g --offload-arch=gfx942:xnack+` (adapt the architecture). Frameworks need ASan builds of their GPU libraries for full coverage, which is costly; on AMD the runtime layers (classification, dropping the culprit, replay) carry more of the protection. Without a VALIDATION_GPU, validation is unavailable and unvalidated images can only run exclusively.

`runtime/validator.py`

```
"""validator.py - image validation inside the runtime.

Runs an image's /app/smoke_test.py under a GPU memory checker on a dedicated
validation GPU (VALIDATION_GPU, excluded from sharing and without MPS). A clean
run approves the image digest; only approved digests may share a GPU.
  NVIDIA: compute-sanitizer from the host CUDA toolkit, mounted into the image.
  AMD:    the submitted image must be an AddressSanitizer build (xnack+).
Host paths are used for volumes because the runtime drives the host's Docker.
"""
import os, re, threading, time, uuid
import docker

APPROVED = "/var/lib/gpushare/approved_digests.txt"
SANITIZER_HOST_DIR = os.environ.get("SANITIZER_DIR", "/usr/local/cuda/compute-sanitizer")
TIMEOUT_S = int(os.environ.get("VALIDATION_TIMEOUT_S", "3600"))
FAILED = re.compile(r"ERROR SUMMARY: [1-9]|AddressSanitizer|Memory access fault", re.I)


class Validator:
    def __init__(self, vendor, gpu, amd_render_minor=None):
        self.vendor, self.gpu, self.minor = vendor, gpu, amd_render_minor
        self.dock = docker.from_env()
        self.jobs = {}
        self.lock = threading.Lock()

    def approved(self, digest):
        try:
            return digest in {l.strip() for l in open(APPROVED)}
        except OSError:
            return False

    def revoke(self, digest):
        with self.lock:
            try:
                keep = [l for l in open(APPROVED) if l.strip() != digest]
                open(APPROVED, "w").writelines(keep)
            except OSError:
                pass

    def submit(self, image):
        if self.gpu is None:
            raise RuntimeError("no VALIDATION_GPU configured on this node")
        job = str(uuid.uuid4())[:8]
        self.jobs[job] = {"image": image, "status": "running", "started": time.time()}
        threading.Thread(target=self._run, args=(job, image), daemon=True).start()
        return job

    def _run(self, job, image):
        j = self.jobs[job]
        try:
            img = self.dock.images.pull(image)
```

```
            digest = (img.attrs.get("RepoDigests") or [img.id])[0]
            j["digest"] = digest
            if self.vendor == "nvidia":
                kw = dict(device_requests=[docker.types.DeviceRequest(
                                device_ids=[str(self.gpu)], capabilities=[["gpu"]])],
                          volumes={SANITIZER_HOST_DIR: {"bind": "/opt/cs", "mode": "ro"}},
                          entrypoint="/opt/cs/compute-sanitizer",
                          command=["--tool", "memcheck", "--error-exitcode", "99",
                                   "--target-processes", "all", "python3", "/app/smoke_test.py"])
            else:
                kw = dict(devices=["/dev/kfd:/dev/kfd",
                                   f"/dev/dri/renderD{self.minor}:/dev/dri/renderD{self.minor}"],
                          environment={"HSA_XNACK": "1", "ASAN_OPTIONS": "abort_on_error=1"},
                          entrypoint="python3", command=["/app/smoke_test.py"])
            c = self.dock.containers.run(image, detach=True, **kw)
            rc = c.wait(timeout=TIMEOUT_S)["StatusCode"]
            log = c.logs().decode(errors="replace")
            c.remove(force=True)
            ok = rc == 0 and not FAILED.search(log)
            j.update(status="approved" if ok else "rejected", exit_code=rc, log_tail=log[-4000:])
            if ok:
                with self.lock, open(APPROVED, "a") as f:
                    f.write(digest + "\n")
        except Exception as e:
            j.update(status="error", error=str(e))
        j["ended"] = time.time()
```

8. Test and acceptance plan

The test scripts use ctypes against the driver (libcuda) or the HIP runtime (libamdhip64 with hiprtc), so they need neither PyTorch nor a toolkit. Run them in order; each step isolates one mechanism before the next adds another.

#	Test	Command	Pass criterion
T1	Full compute inventory	NVIDIA: CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=100 python3 mps_sm_count.py · AMD: setup_amd_node.sh	Matches the GPU specification
T2	Compute ceiling visible	NVIDIA: python3 mps_sm_count.py 10 25 50 100	SM count scales with % (TPC rounding)
T3	Compute ceiling effective	python3 mps_burn.py 25 50 25 · python3 hip_burn.py 25 50 25	ms/launch ≈ inversely proportional to share (±15 %)
T4	Memory ceiling	mem_cap_test.py nvidia|amd with a 2 GiB cap	Total within one 256 MiB block of the cap
T5	Container path	manual docker run of §4.3	T2 and T4 identical inside a container
T6	IWMR start-up	run_runtime.sh; GET /state	All GPUs listed, healthy; validation GPU excluded
T7	Validation	POST /images/validate with a clean and a faulty image	Clean → approved; faulty → rejected
T8	Admission and placement	submit workloads until GPUs are full	Best-fit order; never over-committed; 503 with reasons
T9	Fault blast radius	3 × burn clients, then fault_inject.py on the same GPU	Record which clients die and their error text
T10	Recovery	repeat T9 with workloads under the IWMR	Culprit dropped and digest revoked; victims ready again; their requests answered
T11	OOM path	oversized batches against a small cap	503 + Retry-After, process alive; on exit: cap ×1.25 and re-placed

Table 7. Acceptance tests.

Fault blast-radius experiment (T9)

```
# NVIDIA: all clients on GPU 0's MPS daemon
export CUDA_MPS_PIPE_DIRECTORY=/var/run/nvidia-mps/gpu0
python3 tests/mps_burn.py 25 25 25 &          # three victims, 10 s each
sleep 3 && python3 tests/fault_inject.py nvidia
wait; journalctl -k --since "-1 min" | grep -i xid

# AMD: disjoint CU sets, no shared server
python3 tests/hip_burn.py 25 25 25 &
sleep 3 && python3 tests/fault_inject.py amd
wait; journalctl -k --since "-1 min" | grep -iE "amdgpu.*(fault|reset)"
```

Expected outcome to confirm: under MPS all three burn clients fail with the injector (their error text defines the victim patterns in guard.py); on AMD only the injector should die unless the fault escalates to a GPU reset.

9. Operations command reference

Task	NVIDIA	AMD
Start the runtime	VENDOR=nvidia ./deploy/run_runtime.sh	VENDOR=amd ./deploy/run_runtime.sh
Runtime logs / restart	docker logs -f iwmr · docker restart iwmr	same
Node state	curl localhost:8080/state	same
Host sharing service	systemctl status nvidia-mps@N	none
List sharing clients	echo get_server_list | nvidia-cuda-mps-control (pipe dir set)	amd-smi process
Kernel fault log	journalctl -k | grep Xid	journalctl -k | grep amdgpu
Return a drained GPU	curl -X POST localhost:8080/gpus/N/undrain	same
Inspect approved digests	cat /var/lib/gpushare/approved_digests.txt	same
Hard partitions	nvidia-smi mig …	amd-smi set --compute-partition / --memory-partition
MPS logs	/var/log/nvidia-mps/gpuN/{control,server}.log	—

Table 8. Command reference.

10. Limitations and items to verify

- **Bandwidth and cache interference** is not isolated by either vendor's fine-grained mechanism and is not mitigated by best-fit placement. Latency-critical workloads should use isolation=exclusive or hardware partitions.
- **MPS fault propagation:** confirm with T9 whether a fault terminates all clients of the server on the GPUs in use; newer architectures contain some error classes.
- **Exit patterns** differ between CUDA and ROCm versions; update guard.py from T9 output.
- **Memory-library coverage:** hipMallocManaged, hipExtMallocWithFlags, hipMallocPitch and symbols resolved through hipGetProcAddress bypass libhipmemcap.so. Repeat T4 with the real framework.
- **CU index layout** on multi-XCC AMD GPUs (Section 5.2).
- **Context overhead** must be measured per GPU model (CTX_OVERHEAD_MB).
- **Kernel hangs** trigger watchdog resets that affect every process on the GPU on both vendors.
- **Single runtime per node:** registry and journal are in memory. An IWMR restart removes all workload containers and loses in-flight requests; clients retry. A persistent store would allow re-adoption.
- **Privilege:** the IWMR holds the Docker socket and host PID namespace. Add authentication (e.g. mutual TLS) before exposing the API outside a trusted network.

- **Culprit attribution on NVIDIA** relies mainly on the client's own error text; a culprit that dies without a recognisable message is classified app_error and, inside an incident, treated as a victim.

Appendix A. Test scripts

gpu_lowlevel.py provides the minimal driver/runtime access used by all tests. mps_sm_count.py and mps_burn.py are the scripts developed earlier for MPS; hip_burn.py is their AMD counterpart with the same structure.

tests/gpu_lowlevel.py

```
"""gpu_lowlevel.py - minimal ctypes access to the CUDA driver and HIP runtime.

Used by the test scripts so they run with no PyTorch and no toolkit:
NVIDIA kernels are PTX (JIT-compiled by the driver); AMD kernels are HIP C++
compiled at runtime with hiprtc (part of every ROCm install).
"""
import ctypes, glob


class Cuda:
    def __init__(self, device=0):
        self.lib = ctypes.CDLL("libcuda.so.1")
        self.call("cuInit", 0)
        self.dev, ctx = ctypes.c_int(), ctypes.c_void_p()
        self.call("cuDeviceGet", ctypes.byref(self.dev), device)
        self.call("cuDevicePrimaryCtxRetain", ctypes.byref(ctx), self.dev)   # MPS client here
        self.call("cuCtxSetCurrent", ctx)

    def call(self, name, *args, check=True):
        rc = getattr(self.lib, name)(*args)
        if rc and check:
            s = ctypes.c_char_p()
            self.lib.cuGetErrorString(rc, ctypes.byref(s))
            raise RuntimeError(f"{name}: {s.value.decode() if s.value else rc}")
        return rc

    def kernel(self, ptx: bytes, name: str):
        mod, fn = ctypes.c_void_p(), ctypes.c_void_p()
        self.call("cuModuleLoadData", ctypes.byref(mod), ctypes.c_char_p(ptx))
        self.call("cuModuleGetFunction", ctypes.byref(fn), mod, name.encode())
        return fn

    def launch(self, fn, blocks, threads, *args):
        params = (ctypes.c_void_p * max(len(args), 1))(*[ctypes.addressof(a) for a in args])
        self.call("cuLaunchKernel", fn, blocks, 1, 1, threads, 1, 1, 0, None, params, None)
        self.call("cuCtxSynchronize")

    def malloc(self, n):
        p = ctypes.c_uint64()
        rc = self.call("cuMemAlloc_v2", ctypes.byref(p), ctypes.c_size_t(n), check=False)
        return p.value if rc == 0 else None


class Hip:
    def __init__(self, device=0):
        self.lib = ctypes.CDLL("libamdhip64.so")
        self.rtc = None
        self.call("hipSetDevice", device)

    def call(self, name, *args, check=True):
        rc = getattr(self.lib, name)(*args)
        if rc and check:
            self.lib.hipGetErrorString.restype = ctypes.c_char_p
            raise RuntimeError(f"{name}: {self.lib.hipGetErrorString(rc).decode()}")
        return rc

    def kernel(self, src: str, name: str):
        if self.rtc is None:
            self.rtc = ctypes.CDLL("libhiprtc.so")
        prog, size = ctypes.c_void_p(), ctypes.c_size_t()
        self.rtc.hiprtcCreateProgram(ctypes.byref(prog), src.encode(), b"k.cpp", 0, None, None)
        if self.rtc.hiprtcCompileProgram(prog, 0, None):
            raise RuntimeError("hiprtc compilation failed")
        self.rtc.hiprtcGetCodeSize(prog, ctypes.byref(size))
        code = ctypes.create_string_buffer(size.value)
        self.rtc.hiprtcGetCode(prog, code)
        mod, fn = ctypes.c_void_p(), ctypes.c_void_p()
        self.call("hipModuleLoadData", ctypes.byref(mod), code)
        self.call("hipModuleGetFunction", ctypes.byref(fn), mod, name.encode())
        return fn

    def launch(self, fn, blocks, threads, *args):
        params = (ctypes.c_void_p * max(len(args), 1))(*[ctypes.addressof(a) for a in args])
```

```
        self.call("hipModuleLaunchKernel", fn, blocks, 1, 1, threads, 1, 1, 0, None, params, None)
        self.call("hipDeviceSynchronize")

    def malloc(self, n):
        p = ctypes.c_void_p()
        rc = self.call("hipMalloc", ctypes.byref(p), ctypes.c_size_t(n), check=False)
        return p.value if rc == 0 else None


def amd_cu_count(index=0):
    """Total CUs of the index-th GPU from KFD topology (no HIP enum dependencies)."""
    gpus = []
    for node in sorted(glob.glob("/sys/class/kfd/kfd/topology/nodes/*"),
                       key=lambda p: int(p.rsplit("/", 1)[1])):
        props = dict(l.split() for l in open(f"{node}/properties") if len(l.split()) == 2)
        if int(props.get("simd_count", 0)):
            gpus.append(int(props["simd_count"]) // int(props["simd_per_cu"]))
    return gpus[index]


def open_gpu(vendor):
    return Cuda() if vendor == "nvidia" else Hip()
```

tests/mps_sm_count.py

```
#!/usr/bin/env python3
"""Print the SM count visible to this process (reflects the MPS active thread percentage).

Usage:
  python mps_sm_count.py                # SMs visible now
  python mps_sm_count.py 25 50 100      # SMs at each CUDA_MPS_ACTIVE_THREAD_PERCENTAGE
"""
import ctypes, os, subprocess, sys

def sm_count(device=0):
    cu = ctypes.CDLL("libcuda.so.1")
    cu.cuInit(0)
    dev, ctx, sms = ctypes.c_int(), ctypes.c_void_p(), ctypes.c_int()
    cu.cuDeviceGet(ctypes.byref(dev), device)
    cu.cuDevicePrimaryCtxRetain(ctypes.byref(ctx), dev)  # connect as MPS client
    cu.cuCtxSetCurrent(ctx)
    cu.cuDeviceGetAttribute(ctypes.byref(sms), 16, dev)  # 16 = MULTIPROCESSOR_COUNT
    return sms.value

if __name__ == "__main__":
    if len(sys.argv) == 1:
        print(sm_count())
    else:
        for pct in sys.argv[1:]:
            env = dict(os.environ, CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=pct)
            out = subprocess.run([sys.executable, __file__], env=env,
                                 capture_output=True, text=True).stdout.strip()
            print(f"{pct:>3}%  {out} SMs")
```

tests/mps_burn.py

```
#!/usr/bin/env python3
"""Dummy GPU workload for MPS testing (no torch needed).

Usage:  python mps_burn.py 25 50 25   # 3 concurrent clients at 25%, 50%, 25% SMs
"""
import ctypes, json, os, subprocess, sys, time

# GPU kernel: each thread loops ITERS times doing x = x*1 + 1 (dummy load)
PTX = rb"""
.version 7.0
.target sm_52
.address_size 64
.visible .entry burn(.param .u64 out, .param .u32 iters)
{
  .reg .f32 %f<3>; .reg .u32 %r<4>; .reg .u64 %rd<2>; .reg .pred %p;
  ld.param.u64 %rd1, [out]; ld.param.u32 %r1, [iters];
  mov.u32 %r2, %tid.x; cvt.rn.f32.u32 %f1, %r2;
  mov.f32 %f2, 0f3F800000; mov.u32 %r3, 0;
LOOP:
  fma.rn.f32 %f1, %f1, %f2, %f2;
  add.u32 %r3, %r3, 1;
  setp.lt.u32 %p, %r3, %r1;
  @%p bra LOOP;
  setp.eq.f32 %p, %f1, 0fBF800000;
  @%p st.global.f32 [%rd1], %f1;
  ret;
```

```
}
"""
DURATION, ITERS, BLOCKS, THREADS = 10, 20000, 1024, 256


def worker():
    """One MPS client: run the kernel repeatedly for DURATION seconds."""
    cu = ctypes.CDLL("libcuda.so.1")
    dev, ctx, mod, fn = ctypes.c_int(), ctypes.c_void_p(), ctypes.c_void_p(), ctypes.c_void_p()
    sms, ptr = ctypes.c_int(), ctypes.c_uint64()

    cu.cuInit(0)
    cu.cuDeviceGet(ctypes.byref(dev), 0)
    cu.cuDevicePrimaryCtxRetain(ctypes.byref(ctx), dev)          # becomes MPS client here
    cu.cuCtxSetCurrent(ctx)
    cu.cuDeviceGetAttribute(ctypes.byref(sms), 16, dev)          # 16 = SM count
    cu.cuModuleLoadData(ctypes.byref(mod), ctypes.c_char_p(PTX)) # compile kernel
    cu.cuModuleGetFunction(ctypes.byref(fn), mod, b"burn")
    cu.cuMemAlloc_v2(ctypes.byref(ptr), 4)

    out, iters = ctypes.c_uint64(ptr.value), ctypes.c_uint32(ITERS)
    args = (ctypes.c_void_p * 2)(ctypes.addressof(out), ctypes.addressof(iters))

    launches, t0 = 0, time.time()
    while time.time() - t0 < DURATION:
        cu.cuLaunchKernel(fn, BLOCKS, 1, 1, THREADS, 1, 1, 0, None, args, None)
        cu.cuCtxSynchronize()
        launches += 1
    ms = 1000 * (time.time() - t0) / max(launches, 1)

    print(json.dumps({"pct": os.environ.get("CUDA_MPS_ACTIVE_THREAD_PERCENTAGE", "100"),
                      "sms": sms.value, "launches": launches, "ms": round(ms, 2)}))


def launch(percentages):
    """Start one client per percentage, all at once, and print their results."""
    procs = []
    for pct in percentages:
        env = dict(os.environ, CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=pct)
        procs.append(subprocess.Popen([sys.executable, __file__, "--worker"],
                                      env=env, stdout=subprocess.PIPE, text=True))

    print(" pct  SMs  launches  ms/launch")
    for p in procs:
        lines = p.communicate()[0].strip().splitlines()
        if not lines:
            print("client failed")
            continue
        r = json.loads(lines[-1])
        print(f"{r['pct']:>3}%  {r['sms']:>3}  {r['launches']:>8}  {r['ms']:>9}")


if __name__ == "__main__":
    if "--worker" in sys.argv:
        worker()
    else:
        launch(sys.argv[1:] or ["100"])
```

tests/hip_burn.py

```
#!/usr/bin/env python3
"""hip_burn.py - AMD counterpart of mps_burn.py (same two-function structure).

Each client gets a disjoint CU range via HSA_CU_MASK instead of an MPS percentage.

  python3 hip_burn.py 25 50 25     # 3 concurrent clients on 25 %, 50 %, 25 % of the CUs
"""
import ctypes, json, math, os, subprocess, sys, time
from gpu_lowlevel import Hip, amd_cu_count

SRC = r"""
extern "C" __global__ void burn(float *out, unsigned iters) {
  float x = threadIdx.x, c = 1.0f;
  for (unsigned i = 0; i < iters; ++i) x = x * c + c;
  if (x == -1.0f) *out = x;          // never true; keeps the loop alive
}
"""
DURATION, ITERS, BLOCKS, THREADS = 10, 20000, 1024, 256


def worker():
    gpu = Hip()
    fn = gpu.kernel(SRC, "burn")
```

```
    p = ctypes.c_void_p()
    gpu.call("hipMalloc", ctypes.byref(p), ctypes.c_size_t(4))
    out, iters = ctypes.c_void_p(p.value), ctypes.c_uint32(ITERS)
    launches, t0 = 0, time.time()
    while time.time() - t0 < DURATION:
        gpu.launch(fn, BLOCKS, THREADS, out, iters)
        launches += 1
    ms = 1000 * (time.time() - t0) / max(launches, 1)
    print(json.dumps({"mask": os.environ.get("HSA_CU_MASK", "all"),
                      "launches": launches, "ms": round(ms, 2)}))


def launch(percentages):
    total, start, procs = amd_cu_count(), 0, []
    for pct in percentages:
        n = max(2, math.ceil(int(pct) / 100 * total))
        n += n % 2
        if start + n > total:
            sys.exit(f"shares exceed the {total} CUs of the GPU")
        env = dict(os.environ, HSA_CU_MASK=f"0:{start}-{start + n - 1}")
        start += n                                      # disjoint ranges
        procs.append(subprocess.Popen([sys.executable, __file__, "--worker"],
                                      env=env, stdout=subprocess.PIPE, text=True))
    print(f"GPU has {total} CUs\n mask          launches  ms/launch")
    for p in procs:
        lines = p.communicate()[0].strip().splitlines()
        if not lines:
            print("client failed")
            continue
        r = json.loads(lines[-1])
        print(f"{r['mask']:<13} {r['launches']:>8}  {r['ms']:>9}")


if __name__ == "__main__":
    if "--worker" in sys.argv:
        worker()
    else:
        launch(sys.argv[1:] or ["100"])
```

tests/mem_cap_test.py

```
#!/usr/bin/env python3
"""mem_cap_test.py - verify the per-workload memory cap is enforced.

Allocates 256 MiB blocks until an allocation fails and prints the total reached.
Expect a total close to the cap (MPS: CUDA_MPS_PINNED_DEVICE_MEM_LIMIT;
AMD: HIPMEM_LIMIT_BYTES with LD_PRELOAD=libhipmemcap.so).

  NVIDIA: CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0=2G python3 mem_cap_test.py nvidia
  AMD:    LD_PRELOAD=/opt/gpushare/lib/libhipmemcap.so HIPMEM_LIMIT_BYTES=2147483648 \
          python3 mem_cap_test.py amd
"""
import sys
from gpu_lowlevel import open_gpu

BLOCK = 256 * 2**20
vendor = sys.argv[1] if len(sys.argv) > 1 else "nvidia"
gpu = open_gpu(vendor)
total, ptrs = 0, []
while total < 512 * 2**30:
    p = gpu.malloc(BLOCK)
    if p is None:
        break
    ptrs.append(p)
    total += BLOCK
print(f"allocated {total / 2**30:.2f} GiB before the first failure ({len(ptrs)} blocks)")
```

tests/fault_inject.py

```
#!/usr/bin/env python3
"""fault_inject.py - deliberately trigger an illegal GPU memory access.

Use only on the testbed. Start several burn clients first, then run this as an
extra client on the same GPU and record which processes die and with which
error. That measures the fault blast radius of each backend.

  python3 fault_inject.py nvidia     (run with the same MPS pipe dir as the victims)
  python3 fault_inject.py amd
"""
import ctypes, sys, time
from gpu_lowlevel import open_gpu
```

```
PTX = rb"""
.version 7.0
.target sm_52
.address_size 64
.visible .entry boom()
{
  .reg .u64 %rd1; .reg .f32 %f1;
  mov.u64 %rd1, 16;                 // address 0x10 is never mapped
  mov.f32 %f1, 0f3F800000;
  st.global.f32 [%rd1], %f1;
  ret;
}
"""
HIP_SRC = r"""
extern "C" __global__ void boom() { *(volatile int*)0x10 = 1; }
"""

vendor = sys.argv[1] if len(sys.argv) > 1 else "nvidia"
gpu = open_gpu(vendor)
fn = gpu.kernel(PTX if vendor == "nvidia" else HIP_SRC, "boom")
print(f"[{time.strftime('%H:%M:%S')}] launching faulting kernel", flush=True)
try:
    gpu.launch(fn, 1, 1)
    print("no fault raised (unexpected)")
except RuntimeError as e:
    print(f"fault reported to the culprit: {e}", flush=True)
    sys.exit(1)
```